Anatomía de un LLM

Fundamentos matemáticos de los modelos de lenguaje

Fase V · Alineamiento, aprendizaje por refuerzo, agentes e interpretabilidad

Módulo 20

Ajuste fino e instrucciones

Ajuste supervisado, olvido catastrófico y adaptación eficiente en parámetros

---

Apuntes del curso

Edición de octubre de 2026

Presentación del módulo

Un modelo preentrenado con la pérdida de los módulos 15 y 16 es un *modelo base*: dado un texto, lo continúa con la distribución del corpus de entrenamiento. Ante la entrada «p: capital de francia\n r:», el mini-GPT de la práctica produce una continuación plausible del corpus y no la respuesta «paris», aunque ese dato aparezca en sus datos de entrenamiento. Lo que convierte un modelo base en un asistente es el *posentrenamiento*: primero se ajusta el modelo para imitar respuestas de calidad (ajuste supervisado o SFT, este módulo) y después se optimizan las preferencias humanas (módulo 21). Este módulo estudia el ajuste fino como problema de optimización que parte de un modelo ya entrenado: la transferencia y sus técnicas clásicas, la pérdida del ajuste supervisado con instrucciones, el olvido de lo aprendido y su control, y los métodos que modifican solo una fracción mínima de los parámetros, con LoRA y QLoRA como casos centrales. Sobre los procedimientos de Claude, Gemini y Grok solo se usa información pública; sus laboratorios no publican los datos ni las recetas de su ajuste supervisado.

Objetivos

Al finalizar el módulo, el estudiante debe ser capaz de:

1. formular el ajuste fino como minimización de una pérdida de destino a partir de un modelo preentrenado, distinguirlo de la extracción de características y aplicar las técnicas de ULMFiT (tasas discriminativas, tasa triangular inclinada y descongelación gradual);
2. definir la pérdida enmascarada del ajuste supervisado, demostrar que es la log-verosimilitud condicional de la respuesta, construir la plantilla de chat y razonar sobre normalización, relleno, empaquetado y acumulación de gradientes;
3. describir el olvido catastrófico, resolver el modelo cuadrático de compromiso entre tarea nueva y antigua y deducir la penalización de EWC a partir de la aproximación de Laplace y la información de Fisher;
4. definir los métodos de ajuste eficiente en parámetros (adaptadores, ajuste de *prompt* y de prefijo), contar sus parámetros y explicar la dimensión intrínseca del ajuste fino;
5. formular LoRA, contar sus parámetros, demostrar la cota de rango y las fórmulas de sus gradientes, justificar la inicialización $B = 0$ y la escala α/r y relacionarla con el teorema de Eckart–Young;
6. explicar la cuantización NF4 y la doble cuantización de QLoRA, estimar la memoria de cada configuración de ajuste y decidir entre ajuste fino, instrucciones en el contexto y recuperación de documentos.

Requisitos previos

Del módulo 1 se usa la regla de la cadena de probabilidades y la entropía cruzada; del módulo 3, la SVD y el teorema de Eckart–Young; del módulo 4, los gradientes matriciales; del módulo 5, la distribución normal y el cálculo de esperanzas; del módulo 7, el descenso de gradiente y Adam; del módulo 12, los tokens especiales; de los módulos 13 a 15, la atención y el mini-GPT; del módulo 17, la contabilidad de memoria del entrenamiento (16 bytes por parámetro), y del módulo 19, la cuantización por bloques.

Organización

Tema	Contenido	Fuentes principales
1	Transferencia y ajuste fino	[19], [17], [28], [5]
2	Ajuste supervisado con instrucciones	[19], [6], [27], [25], [29]
3	Olvido catastrófico	[24], [20], [3], [14]
4	Ajuste eficiente en parámetros	[16], [21], [23], [1]
5	LoRA: adaptación de bajo rango	[18], [12], [10], [5]
6	QLoRA y elección de estrategia	[11], [7], [19]

Cada tema termina con una lista de lecturas y con ejercicios graduados en tres niveles: $\star$ comprensión, $\star\star$ cálculo y $\star\star\star$ demostración. Los ejercicios marcados con *Fuente* están adaptados de cursos y libros de texto universitarios; el enunciado se ha redactado de nuevo y, en algunos casos, se han cambiado los datos para que puedan resolverse a mano. Las soluciones completas, paso a paso, están en el apéndice A.

Sobre las fuentes

El texto es de redacción propia. La organización sigue el capítulo 8, «Post-training», de Jurafsky y Martin [19] (§8.1 sobre el ajuste con instrucciones y §8.2 sobre el ajuste eficiente en parámetros), las clases 8 («Post-training») y 9 («Efficient Adaptation») del curso CS224N de Stanford [5], la clase 15 («Alignment – SFT/RLHF») del curso CS336 de Stanford [6] y el temario del curso 11-711 de Carnegie Mellon [4]. De la tarea 5 de CS336 [8, 9] proceden los ejercicios sobre máscaras, normalización y empaquetado. La transferencia clásica sigue a Howard y Ruder [17] y el §14.2 de Zhang *et al.* [28]; el olvido catastrófico, a McCloskey y Cohen [24] y Kirkpatrick *et al.* [20], con la aproximación de Laplace de Bishop [3, §4.4] y el análisis cuadrático de la penalización L^2 de Goodfellow, Bengio y Courville [14, §7.1.1]. Los métodos se toman de sus artículos originales: FLAN [27], T0 [26], InstructGPT [25], LIMA [29], los adaptadores [16], el ajuste de *prompt* [21] y de prefijo [23], la dimensión intrínseca [22, 1], LoRA [18] y QLoRA [11]. La relación con la SVD remite al módulo 3 y a Deisenroth, Faisal y Ong [10, cap. 4]. Sobre los modelos comerciales (Claude, Gemini, Grok) solo se usa información pública.

Notación

Símbolo	Significado
θ_0, θ	parámetros del modelo preentrenado y parámetros ajustados
$u = u_{1:n}$, $y = y_{1:m}$	instrucción (*prompt*) y respuesta, como secuencias de tokens
$x_{1:T}$, $m_t \in \{0, 1\}$	secuencia completa (instrucción seguida de respuesta) y máscara de las posiciones que puntúan
$\mathcal{L}_{\text{SFT}}$	pérdida del ajuste supervisado (entropía cruzada enmascarada)
η, η^l	tasa de aprendizaje global y de la capa l
λ	intensidad de la penalización que ancla los parámetros a θ_0
$\mathcal{I}(\theta)$, F_i	matriz de información de Fisher y su diagonal
D, d'	número total de parámetros y dimensión del subespacio entrenable (tema 4)
$W \in \mathbb{R}^{d\times k}$, $x \in \mathbb{R}^k$	en los temas 4 a 6, matriz preentrenada congelada y entrada de la capa
$A \in \mathbb{R}^{r\times k}$, $B \in \mathbb{R}^{d\times r}$	factores de LoRA; r es el rango
α, $s = \alpha/r$	hiperparámetro de escala de LoRA y factor efectivo
$\sigma_1 \geq \sigma_2 \geq \cdots$	valores singulares
Φ, Φ^{-1}	función de distribución de la normal estándar y su inversa (función cuantil)
N, N_{LoRA}	parámetros del modelo base y de los adaptadores

Índice

Presentación del módulo **ii**

1 Transferencia y ajuste fino **1**

1.1 Modelo base, tarea de origen y tarea de destino . . . 1
1.2 Extracción de características y ajuste completo . . . 1
1.3 ULMFiT: ajuste fino de un modelo de lenguaje . . . 2
1.4 Del ajuste por tareas al posentrenamiento . . . 3
Ejercicios del tema 1 . . . 4

2 Ajuste supervisado con instrucciones **5**

2.1 Datos de instrucciones . . . 5
2.2 La pérdida enmascarada . . . 6
2.3 Lotes: relleno, normalización, empaquetado y acumulación . . . 7
2.4 La plantilla de chat . . . 8
2.5 ¿Cuántos datos hacen falta? La alineación superficial . . . 9
Ejercicios del tema 2 . . . 9

3 Olvido catastrófico **11**

3.1 El fenómeno . . . 11
3.2 Un modelo cuadrático del compromiso . . . 11
3.3 Consolidación elástica de pesos . . . 12
3.3.1 Justificación bayesiana . . . 12
3.3.2 Información de Fisher . . . 13
3.4 Otros remedios . . . 13
Ejercicios del tema 3 . . . 14

4 Ajuste eficiente en parámetros **16**

4.1 Planteamiento . . . 16
4.2 Adaptadores . . . 17
4.3 Ajuste de *prompt* y de prefijo . . . 17
4.4 Dimensión intrínseca . . . 18
Ejercicios del tema 4 . . . 19

5 LoRA: adaptación de bajo rango **21**
5.1 Definición y número de parámetros . . . 21
5.2 Rango de la corrección . . . 22
5.3 Gradientes e inicialización . . . 23
5.4 La escala α/r . . . 23
5.5 Por qué funciona: relación con la SVD . . . 23
Ejercicios del tema 5 . . . 24

6 QLoRA y elección de estrategia **26**
6.1 Cuantización por bloques . . . 26
6.2 El formato NF4 . . . 26
6.3 Doble cuantización . . . 27
6.4 QLoRA . . . 27
6.5 ¿Ajustar, dar instrucciones en el contexto o recuperar documentos? . . . 28
Ejercicios del tema 6 . . . 29

A Soluciones de los ejercicios **31**
A.1 Tema 1 . . . 31
A.2 Tema 2 . . . 33
A.3 Tema 3 . . . 36
A.4 Tema 4 . . . 38
A.5 Tema 5 . . . 40
A.6 Tema 6 . . . 43

Bibliografía **47**

Tema 1

Transferencia y ajuste fino

Este tema formula el ajuste fino como un problema de optimización que parte de un modelo ya entrenado y presenta las técnicas clásicas de transferencia que preceden al posentrenamiento de los modelos de lenguaje actuales. La distinción entre extracción de características y ajuste completo sigue el §14.2 de Zhang *et al.* [28]; las técnicas de ULMFiT se toman del artículo de Howard y Ruder [17, §3.2 y §3.3], y la visión del posentrenamiento, del capítulo 8 de Jurafsky y Martin [19] y de la clase 8 de CS224N [5].

1.1 Modelo base, tarea de origen y tarea de destino

Definición 1.1 (Aprendizaje por transferencia). Sea f_θ una familia de modelos con parámetros $\theta \in \mathbb{R}^D$. Se llama *tarea de origen* a la minimización de una pérdida $\mathcal{L}_{\text{orig}}$ sobre un conjunto de datos grande, y *modelo preentrenado* a unos parámetros θ_0 que la minimizan aproximadamente. Dada una *tarea de destino* con datos $\mathcal{D}$ y pérdida $\mathcal{L}_{\mathcal{D}}$, el *ajuste fino* consiste en minimizar aproximadamente $\mathcal{L}_{\mathcal{D}}$ mediante un método iterativo de primer orden inicializado en θ_0, normalmente con pocos pasos y una tasa de aprendizaje pequeña.

En los modelos de lenguaje, la tarea de origen es la predicción del token siguiente sobre un corpus masivo (módulos 15 y 16), y θ_0 define un *modelo base*.

Definición 1.2 (Modelo base). Un *modelo base* es un modelo de lenguaje autorregresivo entrenado únicamente con la entropía cruzada del token siguiente sobre texto general. Para un contexto $x_{1:t-1}$, devuelve una aproximación de la distribución condicional del corpus, $p_{\theta_0}(x_t \mid x_{1:t-1})$.

Ejemplo 1.3 (El modelo base no responde). En la práctica del módulo, un mini-GPT preentrenado sobre frases del tipo «roma es la capital de italia» recibe la entrada «p: capital de francia\n r:» y continúa con un fragmento de texto del corpus en lugar de responder «paris». El dato está en los pesos, pero asociado a frases del corpus, no a un turno de respuesta; además, el modelo nunca ha recibido como objetivo un token de fin de turno, de modo que no sabe dónde parar. Lo que le falta es el formato, no el conocimiento (tema 2).

Observación 1.4. Como el problema de destino no es convexo y la optimización es corta, el resultado depende de θ_0 no solo como punto de partida: el método iterativo permanece cerca de θ_0 y actúa como una regularización implícita. El tema 3 hace explícita esta regularización mediante una penalización de la distancia a θ_0.

1.2 Extracción de características y ajuste completo

Sea $\phi_\theta(x) \in \mathbb{R}^h$ la representación que la red asigna a una entrada (por ejemplo, el vector oculto de la última capa) y sea una *cabeza* lineal $z = V\phi_\theta(x) + b$, con $V \in \mathbb{R}^{K\times h}$, que produce

los logits de K clases.

Definición 1.5 (Extracción de características y ajuste completo). En la *extracción de características* (o *sonda lineal*) se fijan los parámetros del cuerpo en $\theta = \theta_0$ y solo se entrenan (V, b). En el *ajuste completo* se entrenan a la vez θ, inicializado en θ_0, y la cabeza, inicializada al azar.

Proposición 1.6 (La sonda lineal es convexa). *Con el cuerpo congelado, la pérdida de entropía cruzada media*

$$J(V, b) = \frac{1}{n} \sum_{i=1}^{n} \Big(\mathrm{LSE}\big(V\phi_{\theta_0}(x_i) + b\big) - \big(V\phi_{\theta_0}(x_i) + b\big)_{y_i} \Big)$$

es una función convexa de (V, b).

Demostración. Para cada i, el vector $\phi_i = \phi_{\theta_0}(x_i)$ es constante, de modo que $z_i = V\phi_i + b$ es una función afín de (V, b). La función $\mathrm{LSE}(z) = \ln \sum_k e^{z_k}$ es convexa (su hessiano es $\mathrm{diag}(p) - pp^\top$, semidefinido positivo, con $p = \mathrm{softmax}(z)$; módulo 8, tema 4), y la composición de una función convexa con una aplicación afín es convexa. El término $-(z_i)_{y_i}$ es lineal en (V, b). Cada sumando es convexo y la media de funciones convexas es convexa. ■

En el ajuste completo, ϕ_θ depende de θ de forma no lineal y la convexidad se pierde. A cambio, el cuerpo puede adaptar sus representaciones a la tarea. La receta habitual [28, §14.2] usa una tasa de aprendizaje pequeña en el cuerpo, que ya está cerca de una buena solución, y una mayor (por ejemplo, diez veces mayor) en la cabeza, que parte de cero.

1.3 ULMFiT: ajuste fino de un modelo de lenguaje

Howard y Ruder [17] propusieron en 2018 el primer procedimiento general para ajustar un modelo de lenguaje preentrenado a tareas de clasificación de textos. Consta de tres etapas: preentrenamiento del modelo de lenguaje en un corpus general, ajuste del modelo de lenguaje al texto del dominio de destino y ajuste del clasificador. Introduce tres técnicas para que el ajuste no destruya lo aprendido.

Definición 1.7 (Tasas de aprendizaje discriminativas). Sea una red de L capas con parámetros $\theta^1, \ldots, \theta^L$. El *ajuste discriminativo* asigna a cada capa su propia tasa, $\theta^l_t = \theta^l_{t-1} - \eta^l \nabla_{\theta^l} J$, con $\eta^{l-1} = \eta^l / 2{,}6$: cada capa aprende 2,6 veces más despacio que la siguiente. La tasa η^L de la última capa se elige ajustando solo esa capa [17, §3.2].

Por inducción, $\eta^l = \eta^L / 2{,}6^{\,L-l}$. La justificación es que las capas inferiores capturan información más general, que conviene modificar menos.

Definición 1.8 (Tasa triangular inclinada). Sean T el número total de iteraciones, $\mathrm{cf} \in (0, 1)$ la fracción de calentamiento, $\rho > 1$ el cociente entre la tasa máxima y la mínima y $\eta_{\max}$ la tasa máxima. Con $c = \lfloor T\,\mathrm{cf} \rfloor$, la *tasa triangular inclinada* (STLR) en la iteración t es

$$p_t = \begin{cases} t/c, & t < c, \\ 1 - \dfrac{t - c}{c\,(1/\mathrm{cf} - 1)}, & t \geq c, \end{cases} \qquad \eta_t = \eta_{\max}\, \frac{1 + p_t(\rho - 1)}{\rho}.$$

Los valores por defecto del artículo son $\mathrm{cf} = 0{,}1$, $\rho = 32$ y $\eta_{\max} = 0{,}01$ [17, §3.2].

Proposición 1.9 (Forma de la tasa triangular). *Supóngase que T cf es entero, de modo que $c = T$ cf. Entonces:*

(i) η_t *es una función afín de* p_t*, creciente, con* $\eta = \eta_{\max}/\rho$ *si* $p_t = 0$ *y* $\eta = \eta_{\max}$ *si* $p_t = 1$*;*

(ii) p_t *crece linealmente de* 0 *a* 1 *en* $[0, c]$ *y decrece linealmente de* 1 *a* 0 *en* $[c, T]$*;*

(iii) en consecuencia, la tasa sube en c *iteraciones desde* $\eta_{\max}/\rho$ *hasta* $\eta_{\max}$ *y baja en* $T - c$ *iteraciones hasta* $\eta_{\max}/\rho$*.*

Demostración. (i) $\eta_t = \frac{\eta_{\max}}{\rho} + \frac{\eta_{\max}(\rho-1)}{\rho}\, p_t$, con pendiente positiva porque $\rho > 1$; los dos valores se obtienen sustituyendo $p_t = 0$ y $p_t = 1$. (ii) En $[0, c)$, $p_t = t/c$ es lineal, vale 0 en $t = 0$ y tiende a 1 cuando $t \to c$. En $[c, T]$, p_t es afín en t con pendiente negativa, vale 1 en $t = c$ y, como $c(1/\text{cf} - 1) = T - c$, vale $1 - (T - c)/(T - c) = 0$ en $t = T$. (iii) Se sigue de (i) y (ii). ■

Definición 1.10 (Descongelación gradual). En la *descongelación gradual* se entrena primero solo la última capa durante una época; después se descongela la capa anterior y se entrenan todas las descongeladas otra época, y así sucesivamente hasta entrenar todas las capas [17, §3.3].

Ejemplo 1.11 (Tasas discriminativas). Con $L = 3$ capas y $\eta^3 = 0{,}01$, la definición 1.7 da $\eta^2 = 0{,}01/2{,}6 = 3{,}846 \cdot 10^{-3}$ y $\eta^1 = 0{,}01/2{,}6^2 = 0{,}01/6{,}76 = 1{,}479 \cdot 10^{-3}$. La primera capa aprende 6,76 veces más despacio que la última.

Observación 1.12. Las tres técnicas persiguen el mismo fin: evitar que los primeros pasos del ajuste, con gradientes grandes procedentes de una cabeza aleatoria, destruyan las representaciones del modelo preentrenado. Howard y Ruder las presentan como remedio al *olvido catastrófico*, que el tema 3 estudia con detalle. La tasa triangular es, además, un antecedente directo del calentamiento seguido de decaimiento que se usa en el entrenamiento de los Transformer (módulo 7).

1.4 Del ajuste por tareas al posentrenamiento

ULMFiT añade una cabeza de clasificación y produce un modelo para una sola tarea. En los modelos de lenguaje generativos se conserva la cabeza de lenguaje: la tarea de destino se formula también como predicción del token siguiente, pero sobre textos que tienen la forma de una conversación entre un usuario y un asistente. Jurafsky y Martin [19, cap. 8] llaman *posentrenamiento* al conjunto de etapas que siguen al preentrenamiento y lo dividen en el ajuste con instrucciones (§8.1, tema 2) y el aprendizaje a partir de preferencias (§8.3 y §8.4, módulo 21). La receta en tres pasos de InstructGPT [25] (ajuste supervisado, modelo de recompensa y aprendizaje por refuerzo) es la referencia publicada que siguen casi todos los asistentes (figura 1.1).

Observación 1.13 (Información pública sobre modelos comerciales). Anthropic, Google y xAI no publican los datos ni los hiperparámetros del ajuste supervisado de Claude, Gemini y Grok. Lo público es el esquema general. Anthropic describió en 2022 el método *Constitutional AI* [2], que incluye una fase supervisada en la que el modelo critica y revisa sus propias respuestas según una lista de principios y se ajusta sobre las respuestas revisadas, seguida de aprendizaje por refuerzo con retroalimentación de un modelo (módulo 21). El informe técnico de Gemini [13, §6.3] describe, a grandes rasgos, un ajuste supervisado seguido de aprendizaje

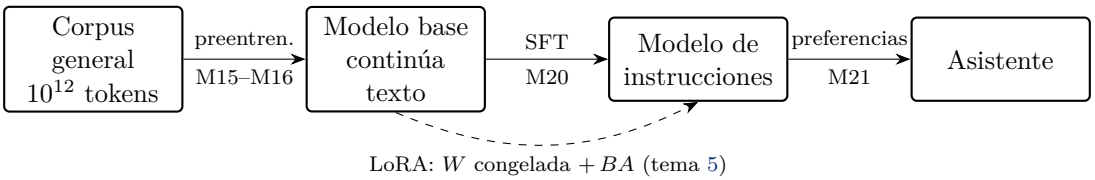


Figura 1.1: Etapas del entrenamiento de un asistente. El ajuste supervisado (SFT) usa entre miles y cientos de miles de diálogos; puede aplicarse a todos los pesos o, con LoRA, solo a una corrección de bajo rango.

por refuerzo con retroalimentación humana. xAI no ha publicado una descripción equivalente del posentrenamiento de Grok.

Lecturas recomendadas para este tema

- Zhang *et al.* [28], §14.2 «Fine-Tuning»: modelo de origen, modelo de destino y tasas distintas para la cabeza.
- Howard y Ruder [17], §3.2 (ajuste discriminativo y tasa triangular) y §3.3 (descongelación gradual).
- Jurafsky y Martin [19], introducción del cap. 8 «Post-training».
- CS224N [5], clase 8 «Post-training (RLHF, SFT, DPO)».
- CMU 11-711 [4], bloque del temario dedicado al ajuste fino.

Ejercicios del tema 1

Ejercicio 1.1 ⋆ Se ajusta por completo una red preentrenada con una cabeza nueva y se aumenta progresivamente la tasa de aprendizaje del cuerpo. Explique cómo cabe esperar que evolucione la exactitud en el conjunto de destino y por qué, relacionando la respuesta con la observación 1.4.

Fuente: adaptado de Zhang *et al.* [28], §14.2, ejercicio 1.

Ejercicio 1.2 ⋆ Justifique si son verdaderas o falsas: (a) la sonda lineal tiene un único mínimo local que es global; (b) el ajuste completo de una red con una capa oculta es un problema convexo; (c) en ULMFiT, la tasa de la primera capa es la mayor.

Ejercicio 1.3 ⋆⋆ Una red tiene $L = 12$ capas y $\eta^{12} = 0{,}01$. Con el ajuste discriminativo de la definición 1.7, calcule η^{11}, η^{10} y η^{1}. ¿Cuál es el menor número de capas L para el que $\eta^{1} < 10^{-6}$ si $\eta^{L} = 0{,}01$?

Ejercicio 1.4 ⋆⋆ Con $T = 1000$ iteraciones y los valores por defecto cf $= 0{,}1$, $\rho = 32$ y $\eta_{\max} = 0{,}01$, calcule la tasa triangular inclinada en $t = 0$, 50, 100, 550 y 1000.

Ejercicio 1.5 ⋆⋆⋆ Considérese la versión continua de la tasa triangular, con $t \in [0, T]$ real y $c = T\,\mathrm{cf}$. Demuestre que su valor medio, $\frac{1}{T}\int_0^T \eta(t)\,dt$, es $\eta_{\max}(\rho+1)/(2\rho)$, independiente de cf, y calcúlelo con los valores por defecto.

Tema 2

Ajuste supervisado con instrucciones

Este tema define el ajuste supervisado (SFT, del inglés *supervised fine-tuning*) de un modelo de lenguaje: la misma entropía cruzada del preentrenamiento, evaluada solo en las posiciones de la respuesta. Se estudian los datos de instrucciones, la pérdida enmascarada y su interpretación probabilística, los detalles de construcción de lotes y la plantilla de chat. Se sigue el §8.1 de Jurafsky y Martin [19], la clase 15 de CS336 [6] y su tarea 5 [8, §4.2], y los artículos de FLAN [27], T0 [26], InstructGPT [25, §3.2 y §3.5] y LIMA [29].

2.1 Datos de instrucciones

Definición 2.1 (Conjunto de instrucciones). Un *conjunto de instrucciones* es una colección de pares $\{(u^{(i)}, y^{(i)})\}_{i=1}^{n}$ en la que $u^{(i)}$ es una instrucción en lenguaje natural, que puede incluir un texto de entrada, e $y^{(i)}$ es la respuesta deseada. El *ajuste con instrucciones* (*instruction tuning*) es el ajuste supervisado sobre un conjunto de instrucciones que abarca muchas tareas distintas.

Los conjuntos se construyen de dos maneras [19, §8.1.1]: reformulando como instrucciones conjuntos de datos supervisados ya existentes, mediante plantillas, o recogiendo demostraciones escritas por personas (o generadas por otros modelos). La colección Flan 2022, por ejemplo, reúne 15 millones de ejemplos de 1836 tareas [19, §8.1.1].

Definición 2.2 (Generalización a tareas no vistas). Se agrupan las tareas en *grupos* (traducción, resumen, inferencia, preguntas y respuestas, etc.). Para medir la generalización a tareas no vistas se ajusta el modelo con todos los grupos salvo uno y se evalúa sin ejemplos (*zero-shot*) en las tareas del grupo excluido.

Con este protocolo, Wei *et al.* [27] ajustaron un modelo de 137 000 millones de parámetros con más de 60 conjuntos de datos expresados mediante plantillas de instrucciones (FLAN). El modelo ajustado superó, sin ejemplos, a GPT-3 de 175 000 millones de parámetros en 20 de los 25 conjuntos evaluados, y el beneficio dependía de la escala: en los modelos más pequeños estudiados, el ajuste con instrucciones empeoraba el resultado en las tareas no vistas. Sanh *et al.* [26] obtuvieron resultados análogos con T0, un modelo de tipo T5 de 11 000 millones de parámetros ajustado con una colección de conjuntos de datos reformulados mediante múltiples *prompts*, que igualaba o superaba en varias pruebas a modelos hasta 16 veces mayores.

Observación 2.3 (El ajuste supervisado en InstructGPT). En InstructGPT [25], el primer paso usa unas 13 000 instrucciones de entrenamiento con demostraciones escritas por anotadores [25, §3.2]. El modelo se ajusta durante 16 épocas con decaimiento coseno de la tasa de aprendizaje y *dropout* residual de 0,2; la pérdida de validación empieza a subir después de la primera época,

pero entrenar más épocas mejora la puntuación del modelo de recompensa y las preferencias humanas [25, §3.5]. La pérdida de validación del SFT no es, por tanto, un indicador fiable de la calidad final.

2.2 La pérdida enmascarada

Sean $u = u_{1:n}$ la instrucción e $y = y_{1:m}$ la respuesta, ambas ya tokenizadas, y sea $x_{1:T} = (u_1, \dots, u_n, y_1, \dots, y_m)$, con $T = n + m$, la secuencia concatenada. El último token de y es el token de fin de turno.

Definición 2.4 (Pérdida del ajuste supervisado). Sea $m_t \in \{0, 1\}$ una *máscara* que vale 1 en las posiciones de la respuesta ($t > n$) y 0 en las de la instrucción. La *pérdida del ajuste supervisado* es la entropía cruzada enmascarada

$$\mathcal{L}_{\mathrm{SFT}}(\theta) = -\frac{1}{\sum_{t=1}^{T} m_t} \sum_{t=1}^{T} m_t \ln p_\theta(x_t \mid x_{1:t-1}).$$

Se divide por el número de posiciones que cuentan, no por T. Si $m_t = 1$ para todo t, se recupera la pérdida del preentrenamiento.

Proposición 2.5 (Interpretación probabilística). *Con la máscara de la definición 2.4,*

$$\mathcal{L}_{\mathrm{SFT}}(\theta) = -\frac{1}{m} \ln p_\theta(y \mid u), \qquad \text{donde} \quad p_\theta(y \mid u) = \prod_{j=1}^{m} p_\theta(y_j \mid u, y_{1:j-1}).$$

En consecuencia, minimizar la suma de $m^{(i)}\mathcal{L}^{(i)}_{\mathrm{SFT}}$ sobre un conjunto de instrucciones equivale a estimar por máxima verosimilitud la distribución condicional de la respuesta dada la instrucción.

Demostración. Como $m_t = 0$ para $t \le n$ y $m_t = 1$ para $t = n + j$, $j = 1, \dots, m$, se tiene $\sum_t m_t = m$ y

$$\sum_{t=1}^{T} m_t \ln p_\theta(x_t \mid x_{1:t-1}) = \sum_{j=1}^{m} \ln p_\theta(y_j \mid u, y_{1:j-1}) = \ln \prod_{j=1}^{m} p_\theta(y_j \mid u, y_{1:j-1}).$$

Por la regla de la cadena (módulo 1, tema 2) aplicada a la distribución condicionada a u, el producto es $p_\theta(y \mid u)$. Dividiendo por m y cambiando el signo se obtiene la igualdad. La suma $\sum_i m^{(i)}\mathcal{L}^{(i)}_{\mathrm{SFT}} = -\sum_i \ln p_\theta(y^{(i)} \mid u^{(i)})$ es la log-verosimilitud negativa condicional del conjunto. ∎

Observación 2.6. Sin máscara se minimizaría $-\frac{1}{T}\ln p_\theta(u, y) = -\frac{1}{T}\big(\ln p_\theta(u) + \ln p_\theta(y \mid u)\big)$: el modelo dedicaría parte de su capacidad a imitar las instrucciones de los usuarios, que no tiene que generar. Este es el sentido de la indicación de Jurafsky y Martin de entrenar solo sobre la respuesta [19, §8.1].

Proposición 2.7 (Gradiente en las posiciones enmascaradas). *Sea $z_t \in \mathbb{R}^K$ el vector de logits de la posición t y $p_t = \mathrm{softmax}(z_t)$. Entonces*

$$\frac{\partial \mathcal{L}_{\mathrm{SFT}}}{\partial z_t} = \frac{m_t}{\sum_s m_s}\,(p_t - e_{x_t}),$$

donde e_{x_t} es el vector canónico del token objetivo. En particular, el gradiente respecto a los logits de una posición enmascarada es nulo.

Demostración. Solo el sumando t depende de z_t. Como $-\ln p_\theta(x_t \mid x_{1:t-1}) = \mathrm{LSE}(z_t) - (z_t)_{x_t}$ y $\nabla \mathrm{LSE}(z) = \mathrm{softmax}(z)$ (módulo 1, tema 4), su gradiente es $p_t - e_{x_t}$. Basta multiplicar por el coeficiente $m_t / \sum_s m_s$. ∎

Observación 2.8. Que el gradiente respecto a los logits de la instrucción sea nulo no implica que la instrucción no influya en el aprendizaje: las posiciones de la respuesta atienden a las de la instrucción, de modo que el gradiente llega a los estados ocultos de la instrucción a través de la atención. Lo que se suprime es la señal que obligaría a *predecir* los tokens de la instrucción.

Ejemplo 2.9 (Con máscara y sin ella). Una secuencia tiene 4 tokens de instrucción y 2 de respuesta, con pérdidas por token $-\ln p$ de 2,0; 1,5; 0,5; 1,0 en la instrucción y 0,4; 0,2 en la respuesta. Sin máscara, la pérdida es $(2{,}0 + 1{,}5 + 0{,}5 + 1{,}0 + 0{,}4 + 0{,}2)/6 = 5{,}6/6 = 0{,}933$; con máscara, $(0{,}4 + 0{,}2)/2 = 0{,}300$. En la práctica del módulo, la muestra «p: doble de 2\n r: 4» tiene 18 posiciones de predicción, de las que solo 3 puntúan (el espacio, «4» y el token de fin): un 16,7 %.

Observación 2.10 (Implementación). En PyTorch no se multiplica por la máscara de forma explícita: se asigna la etiqueta -100 a las posiciones que no cuentan y la función `cross_entropy` con `ignore_index=-100`, el valor por defecto, las excluye del numerador y del denominador de la media.

2.3 Lotes: relleno, normalización, empaquetado y acumulación

Las secuencias de un lote tienen longitudes distintas. Hay dos formas de formar tensores rectangulares.

Definición 2.11 (Relleno y empaquetado). En el *relleno* (*padding*) cada secuencia se completa con un token de relleno hasta la longitud máxima del lote, y la máscara vale 0 en las posiciones de relleno. En el *empaquetado* (*packing*) se concatenan los ejemplos, separados por el token de fin, y la cadena resultante se corta en trozos consecutivos de longitud fija L.

El empaquetado no desperdicia posiciones, pero un trozo puede contener el final de un ejemplo y el principio de otro; la máscara causal permite entonces que un ejemplo atienda al anterior, lo que suele tolerarse porque el token de fin los separa.

Sea un lote de B secuencias con máscaras $m_t^{(i)}$ y pérdidas por token $\ell_t^{(i)} = -\ln p_\theta(x_t^{(i)} \mid x_{1:t-1}^{(i)})$, y sea $M_i = \sum_t m_t^{(i)}$. Hay dos normalizaciones habituales:

$$J_{\mathrm{tok}} = \frac{\sum_i \sum_t m_t^{(i)} \ell_t^{(i)}}{\sum_i M_i}, \qquad J_{\mathrm{sec}} = \frac{1}{B} \sum_{i=1}^{B} \frac{1}{c} \sum_t m_t^{(i)} \ell_t^{(i)},$$

con $c > 0$ una constante fija (por ejemplo, la longitud máxima). En J_{tok} cada secuencia pesa en proporción a la longitud de su respuesta; en J_{sec} todas las secuencias pesan lo mismo por cada token, independientemente del lote en el que caigan. La tarea 5 de CS336 implementa una normalización del segundo tipo: una suma enmascarada dividida por una constante [8, §4.2].

Proposición 2.12 (Acumulación de gradientes). *Divídase un lote de B secuencias en G microlotes $\mathcal{B}_1, \ldots, \mathcal{B}_G$ de B/G secuencias. Sea J_g la pérdida J_{sec} del microlote g calculada*

con tamaño B/G. Si cada microlote retropropaga J_g/G y se suman los gradientes antes de actualizar, el gradiente acumulado coincide con el de J_{sec} sobre el lote completo.

Demostración. Sea $S_i = \frac{1}{c}\sum_t m_t^{(i)}\ell_t^{(i)}$. Entonces $J_g = \frac{G}{B}\sum_{i\in\mathcal{B}_g} S_i$ y

$$\sum_{g=1}^{G}\frac{J_g}{G} = \sum_{g=1}^{G}\frac{1}{B}\sum_{i\in\mathcal{B}_g} S_i = \frac{1}{B}\sum_{i=1}^{B} S_i = J_{\text{sec}},$$

porque los microlotes forman una partición del lote. Por la linealidad del gradiente, $\sum_g \nabla(J_g/G) = \nabla J_{\text{sec}}$. ∎

Observación 2.13. Con la normalización J_{tok} la equivalencia falla en general: cada microlote divide por su propio número de tokens de respuesta y la suma de los cocientes no es el cociente de las sumas, salvo que todos los microlotes tengan el mismo número de tokens que puntúan.

2.4 La plantilla de chat

Un diálogo es una lista de mensajes (r_1, c_1), …, (r_J, c_J), donde $r_j \in \{\text{sistema}, \text{usuario}, \text{asistente}\}$ es el rol y c_j el contenido. Para tratarlo con un modelo autorregresivo hay que convertirlo en una única secuencia.

Definición 2.14 (Plantilla de chat). Sea $\mathcal{V}$ un vocabulario ampliado con un token especial por rol, $\langle r\rangle$, y un token de fin de turno, $\langle\text{fin}\rangle$, que no aparecen en el texto ordinario. La *plantilla de chat* asigna al diálogo la secuencia

$$\langle r_1\rangle\,\tau(c_1)\,\langle\text{fin}\rangle\;\langle r_2\rangle\,\tau(c_2)\,\langle\text{fin}\rangle\;\cdots\;\langle r_J\rangle\,\tau(c_J)\,\langle\text{fin}\rangle,$$

donde τ es el tokenizador. En el ajuste supervisado, la máscara vale 1 en los tokens de $\tau(c_j)$ y en el $\langle\text{fin}\rangle$ que los cierra si $r_j = \text{asistente}$, y 0 en el resto.

tokens	`<|us|>`	`cuánto`	`es`	`2+3`	`<|fin|>`	`<|as|>`	`5`	`<|fin|>`
m_t	0	0	0	0	0	0	1	1

Figura 2.1: Plantilla de chat de un turno (`us`: usuario; `as`: asistente) y su máscara. Solo puntúan el contenido de la respuesta y el token de fin que la cierra (sombreados).

En la inferencia, la aplicación construye la plantilla con todo el historial, añade $\langle\text{asistente}\rangle$ y deja que el modelo genere hasta emitir $\langle\text{fin}\rangle$. Incluir el token de fin en la pérdida es imprescindible para que el modelo aprenda a terminar su turno: en la práctica del módulo, el modelo base no lo emite nunca y, tras 600 pasos de SFT, lo emite en el 99,4 % de los ejemplos de entrenamiento y en el 100 % de los de prueba.

Observación 2.15. Los tokens especiales se añaden al vocabulario antes del ajuste (módulo 12) y sus vectores de *embedding* se aprenden durante él. Por eso la plantilla forma parte del modelo: usar un modelo ajustado con una plantilla distinta de la suya degrada su comportamiento. Cada laboratorio usa su propio formato interno y no siempre lo publica; lo que documentan públicamente las API de Claude, Gemini y Grok es que los mensajes se envían como una lista de pares (rol, contenido).

2.5 ¿Cuántos datos hacen falta? La alineación superficial

El preentrenamiento consume billones de tokens; el ajuste supervisado publicado usa entre miles y cientos de miles de diálogos. Zhou *et al.* [29] ajustaron un modelo LLaMa de 65 000 millones de parámetros con solo 1000 pares instrucción–respuesta seleccionados con cuidado (LIMA); en una evaluación humana controlada, sus respuestas empataron con las de GPT-4 o fueron preferidas a ellas en el 43 % de los casos. De ahí la *hipótesis de la alineación superficial*: casi todo el conocimiento del modelo se adquiere en el preentrenamiento, y el ajuste enseña sobre todo qué formato y qué estilo usar al interactuar con un usuario. En la misma línea, Dettmers *et al.* [11, §1] observaron que un conjunto de 9000 diálogos de alta calidad (OASST1) daba mejores asistentes que una submuestra de 450 000 ejemplos de FLAN v2.

Observación 2.16. Estas conclusiones proceden de evaluaciones concretas de calidad conversacional y no implican que los datos de SFT sean irrelevantes para todas las capacidades: tareas como el razonamiento matemático o la programación se benefician de muchos más ejemplos. La lección robusta es que la calidad y la diversidad de los datos pesan más que su cantidad.

Lecturas recomendadas para este tema

- Jurafsky y Martin [19], §8.1 «Instruction Tuning», con §8.1.1 (datos) y §8.1.2 (evaluación).
- CS336 [6], clase 15 «Alignment – SFT/RLHF», y tarea 5 [8, §4].
- CS224N [5], clase 8 «Post-training».
- Wei *et al.* [27] (FLAN) y Sanh *et al.* [26] (T0).
- Ouyang *et al.* [25], §3.2 (datos) y §3.5 (modelos), y Zhou *et al.* [29].

Ejercicios del tema 2

Ejercicio 2.1 ⋆ Explique por qué un modelo base, al recibir «p: capital de francia\n r:», no responde «paris» aunque el dato esté en su texto de entrenamiento, y qué le enseña el ajuste supervisado.

Ejercicio 2.2 ⋆ ¿Qué ocurre con una posición cuya etiqueta vale -100 en `cross_entropy(…, ignore_index=-100)`? ¿Por qué se usa también en las posiciones de relleno?

Ejercicio 2.3 ⋆⋆ Una secuencia tiene 4 tokens de instrucción con pérdidas 2,1; 1,8; 0,9; 1,1 y 3 de respuesta con 0,6; 0,2; 0,4. Calcule la pérdida media sin máscara y con máscara.

Ejercicio 2.4 ⋆ Se tienen dos ejemplos tokenizados. Ejemplo 1: instrucción $[5, 9, 2]$ y respuesta $[7, 3, 0]$. Ejemplo 2: instrucción $[4, 8]$ y respuesta $[6, 0]$. El índice 0 es el token de fin y el 1, el de relleno. Concatene instrucción y respuesta, rellene a la longitud máxima y construya `input_ids` (la secuencia sin su último token), `labels` (la secuencia sin su primer token) y `response_mask` (1 donde la etiqueta pertenece a la respuesta). ¿Cuántas posiciones puntúan en cada ejemplo?

Fuente: adaptado de Stanford CS336, primavera de 2025, tarea 5 [8, §4.2], problema `tokenize_prompt_and_output`.

Ejercicio 2.5 ⋆⋆ Un microlote tiene 2 secuencias de 5 posiciones, con log-probabilidades por token $(-2{,}0;\ -1{,}5;\ -0{,}5;\ -1{,}0;\ -0{,}1)$ y $(-1{,}2;\ -0{,}4;\ -0{,}2;\ -3{,}0;\ -3{,}0)$ y máscaras de respuesta $(0,0,1,1,1)$ y $(0,1,1,0,0)$. La pérdida del microlote es la suma enmascarada de $-\ln p$ dividida por el tamaño del microlote, por una constante $c = 1$ y por el número de pasos de acumulación, 2. Calcule esa pérdida y compárela con la media por token de respuesta, J_{tok}.

Fuente: adaptado de Stanford CS336, primavera de 2025, tarea 5 [8, §4.2], problemas `masked_normalize` y `sft_microbatch_train_step`.

Ejercicio 2.6 ⋆ Se tienen tres ejemplos de 7, 5 y 9 tokens. Se añade un token de fin tras cada uno, se concatenan y se corta la cadena en trozos consecutivos de 5 tokens, descartando el último si está incompleto. ¿Cuántas secuencias se obtienen y cuántos tokens se pierden? Compárese con rellenar cada ejemplo, con su fin, hasta 10 tokens: ¿qué fracción de posiciones es útil en cada caso?

Fuente: adaptado de Stanford CS336, primavera de 2026, suplemento de la tarea 5 [9, §4.2], problema `data_loading`.

Ejercicio 2.7 ⋆⋆⋆ Un diálogo alterna K turnos de usuario $u^{(1)},\dots,u^{(K)}$ y K respuestas del asistente $y^{(1)},\dots,y^{(K)}$, de longitudes $m_1,\dots,m_K$, en el orden $u^{(1)},y^{(1)},\dots,u^{(K)},y^{(K)}$. Con la máscara de la definición 2.14, demuestre que

$$\mathcal{L}_{\text{SFT}} = -\frac{1}{\sum_k m_k}\sum_{k=1}^{K} \ln p_\theta\big(y^{(k)} \,\big|\, h^{(k)}\big),$$

donde $h^{(k)} = (u^{(1)}, y^{(1)}, \dots, u^{(k)})$ es el historial que precede a la respuesta k (se omiten los tokens de rol y de fin para simplificar). Interprete el resultado.

Tema 3

Olvido catastrófico

Al ajustar todos los pesos con una tarea estrecha, el modelo pierde capacidades que tenía. Este tema define el fenómeno, lo analiza en un modelo cuadrático que puede resolverse a mano y deduce la penalización de la consolidación elástica de pesos (EWC) como aproximación bayesiana. El fenómeno se describió en redes conexionistas en el trabajo de McCloskey y Cohen [24]; EWC procede de Kirkpatrick *et al.* [20]; la aproximación de Laplace sigue el §4.4 de Bishop [3], y el análisis cuadrático de la penalización, el §7.1.1 de Goodfellow, Bengio y Courville [14].

3.1 El fenómeno

Definición 3.1 (Olvido). Sean $\mathcal{L}_A$ la pérdida de una tarea antigua, θ_0 unos parámetros que la resuelven y θ_B los parámetros obtenidos al ajustar θ_0 con una tarea nueva de pérdida $\mathcal{L}_B$. El *olvido* es el aumento $\mathcal{L}_A(\theta_B) - \mathcal{L}_A(\theta_0)$. Se habla de *olvido catastrófico* cuando ese aumento es comparable al que tendría un modelo sin entrenar en la tarea A.

McCloskey y Cohen [24] mostraron en 1989 que una red entrenada secuencialmente en dos tareas pierde casi por completo la primera al aprender la segunda, a diferencia de lo que ocurre en el aprendizaje humano. La causa es que los mismos parámetros codifican ambas tareas y el gradiente de $\mathcal{L}_B$ no tiene en cuenta cuánto importa cada parámetro para A.

Ejemplo 3.2 (El olvido en la práctica del módulo). La práctica mide la pérdida del mini-GPT sobre el texto de preentrenamiento antes y después del ajuste supervisado:

Modelo	Exactitud en prueba	Pérdida de texto (nats)
Base	0	0,192
SFT completo ($\eta = 10^{-3}$)	0,300	2,073
SFT con LoRA ($r = 4$, $\eta = 10^{-2}$)	0,450	5,058
LoRA con el adaptador desactivado	0	0,192
LoRA con repetición de texto	0,475	0,194

La pérdida de texto de los modelos ajustados se dispara. Desactivar el adaptador LoRA devuelve exactamente el modelo base, porque la matriz W no se ha modificado (tema 5). Mezclar en cada paso una fracción de pérdida de preentrenamiento (repetición, definición 3.10) conserva la tarea y casi no olvida.

3.2 Un modelo cuadrático del compromiso

Proposición 3.3 (Muelle hacia el modelo preentrenado). *Sean $a, \theta_0 \in \mathbb{R}$ y $\lambda \geq 0$, y sea*

$$\mathcal{L}(\theta) = (\theta - a)^2 + \lambda(\theta - \theta_0)^2.$$

El único minimizador es $\theta^* = (a + \lambda\theta_0)/(1+\lambda)$*. En él, el olvido medido como* $(\theta^* - \theta_0)^2$ *y el error en la tarea nueva* $(\theta^* - a)^2$ *valen*

$$(\theta^* - \theta_0)^2 = \frac{(a-\theta_0)^2}{(1+\lambda)^2}, \qquad (\theta^* - a)^2 = \frac{\lambda^2(a-\theta_0)^2}{(1+\lambda)^2}.$$

Demostración. $\mathcal{L}'(\theta) = 2(\theta - a) + 2\lambda(\theta - \theta_0)$ se anula si y solo si $\theta(1+\lambda) = a + \lambda\theta_0$. Como $\mathcal{L}'' = 2(1+\lambda) > 0$, $\mathcal{L}$ es estrictamente convexa y el punto crítico es el mínimo global. Además, $\theta^* - \theta_0 = (a-\theta_0)/(1+\lambda)$ y $\theta^* - a = \lambda(\theta_0 - a)/(1+\lambda)$; elevando al cuadrado se obtienen las fórmulas. ■

Corolario 3.4. *Las distancias* $|\theta^* - \theta_0|$ *y* $|\theta^* - a|$ *suman* $|a - \theta_0|$ *para todo* $\lambda \geq 0$*:* θ^* *recorre el segmento entre* a *y* θ_0*, desde* $\theta^* = a$ *(*$\lambda = 0$*, olvido máximo) hasta* $\theta^* \to \theta_0$ *(*$\lambda \to \infty$*, ningún aprendizaje).*

Demostración. Por la proposición, $|\theta^* - \theta_0| + |\theta^* - a| = \frac{1+\lambda}{1+\lambda}|a - \theta_0|$. Los límites se leen en la expresión de θ^*. ■

Ejemplo 3.5. Con $\theta_0 = 1$ y $a = 3$, la proposición 3.3 da $\theta^* = (3+\lambda)/(1+\lambda)$:

λ	θ^*	Olvido $(\theta^* - 1)^2$	Error en la tarea $(\theta^* - 3)^2$
0	3	4	0
1	2	1	1
3	1,5	0,25	2,25

La penalización $\lambda(\theta - \theta_0)^2$ es la energía potencial de un muelle de constante 2λ anclado en θ_0, y θ^* es el punto de equilibrio entre dos fuerzas lineales. El problema real tiene millones de parámetros, y no todos importan igual para la tarea antigua: conviene un muelle más rígido en los parámetros importantes. Esa es la idea de EWC.

3.3 Consolidación elástica de pesos

3.3.1 Justificación bayesiana

Sean $\mathcal{D}_A$ y $\mathcal{D}_B$ los datos de las dos tareas, condicionalmente independientes dado θ. Por el teorema de Bayes,

$$\ln p(\theta \mid \mathcal{D}_A, \mathcal{D}_B) = \ln p(\mathcal{D}_B \mid \theta) + \ln p(\theta \mid \mathcal{D}_A) - \ln p(\mathcal{D}_B \mid \mathcal{D}_A). \tag{3.1}$$

El último término no depende de θ. Toda la información sobre la tarea A está en la distribución a posteriori $p(\theta \mid \mathcal{D}_A)$, que no puede calcularse exactamente. La *aproximación de Laplace* [3, §4.4] la sustituye por una gaussiana centrada en su moda.

Definición 3.6 (Aproximación de Laplace). Sea θ_A un máximo de $g(\theta) = \ln p(\theta \mid \mathcal{D}_A)$ y sea $H_A = -\nabla^2 g(\theta_A)$, definida positiva. La aproximación de Laplace es $p(\theta \mid \mathcal{D}_A) \approx \mathcal{N}(\theta; \theta_A, H_A^{-1})$, que se obtiene del desarrollo de Taylor de segundo orden $g(\theta) \approx g(\theta_A) - \frac{1}{2}(\theta - \theta_A)^\top H_A (\theta - \theta_A)$, en el que el término lineal se anula porque $\nabla g(\theta_A) = 0$.

Sustituyendo en (3.1), maximizar la distribución a posteriori conjunta equivale aproximadamente a minimizar

$$-\ln p(\mathcal{D}_B \mid \theta) + \tfrac{1}{2}(\theta - \theta_A)^\top H_A (\theta - \theta_A).$$

Queda por estimar H_A, una matriz de $D \times D$ con D del orden de millones.

3.3.2 Información de Fisher

Definición 3.7 (Información de Fisher). Sea p_θ una familia de distribuciones derivable en θ. Su *matriz de información de Fisher* es

$$\mathcal{I}(\theta) = \mathbb{E}_{x\sim p_\theta}\big[\nabla_\theta \ln p_\theta(x)\, \nabla_\theta \ln p_\theta(x)^\top\big].$$

El vector $\nabla_\theta \ln p_\theta(x)$ se llama *función de puntuación* (*score*).

$\mathcal{I}(\theta)$ es semidefinida positiva por ser la esperanza de matrices de la forma $vv^\top$. Solo requiere gradientes de primer orden.

Proposición 3.8 (Fisher como hessiano esperado). *Supóngase que se pueden intercambiar derivada e integral. Entonces* $\mathbb{E}_{x\sim p_\theta}[\nabla_\theta \ln p_\theta(x)] = 0$ *y*

$$\mathcal{I}(\theta) = -\mathbb{E}_{x\sim p_\theta}\big[\nabla^2_\theta \ln p_\theta(x)\big].$$

Demostración. Como $\nabla \ln p_\theta = \nabla p_\theta / p_\theta$, se tiene $\mathbb{E}[\nabla \ln p_\theta] = \int \nabla p_\theta(x)\, dx = \nabla \int p_\theta(x)\, dx = \nabla 1 = 0$. Derivando de nuevo, $\nabla^2 \ln p_\theta = \nabla^2 p_\theta / p_\theta - \nabla \ln p_\theta \, \nabla \ln p_\theta^\top$. Tomando esperanzas, el primer término da $\int \nabla^2 p_\theta(x)\, dx = \nabla^2 1 = 0$, de modo que $\mathbb{E}[\nabla^2 \ln p_\theta] = -\mathcal{I}(\theta)$. ∎

Si $\mathcal{D}_A$ consta de n ejemplos y la distribución a priori es plana, $H_A = -\sum_i \nabla^2 \ln p_\theta(x_i)\big|_{\theta_A}$, y por la ley de los grandes números y la proposición 3.8 se aproxima por $n\,\mathcal{I}(\theta_A)$ cuando el modelo describe bien los datos. Kirkpatrick *et al.* se quedan además con la diagonal, $F_i = \mathcal{I}_{ii}(\theta_A)$, que se estima promediando los cuadrados de las componentes del gradiente.

Definición 3.9 (EWC). Sean θ_A los parámetros tras la tarea A y F_i la diagonal de su matriz de Fisher. La *consolidación elástica de pesos* minimiza

$$\mathcal{L}_{\mathrm{EWC}}(\theta) = \mathcal{L}_B(\theta) + \sum_{i=1}^{D} \frac{\lambda}{2} F_i \,(\theta_i - \theta_{A,i})^2,$$

donde $\lambda > 0$ regula la importancia de la tarea antigua [20].

Cada parámetro tiene su propio muelle, de constante λF_i: los parámetros a los que la verosimilitud de la tarea A es sensible (Fisher grande) quedan rígidos y los demás pueden moverse libremente. La proposición 3.3 es el caso de un solo parámetro en el que el coeficiente $\frac{\lambda}{2}F$ de EWC hace el papel del λ de la proposición.

3.4 Otros remedios

Definición 3.10 (Repetición). La *repetición* (*replay*) mezcla en cada paso ejemplos de la tarea antigua: con $\beta \in [0, 1)$, se minimiza $(1-\beta)\mathcal{L}_B(\theta) + \beta\, \mathcal{L}_A(\theta)$.

Otros remedios habituales son una tasa de aprendizaje pequeña y pocas épocas, las técnicas de ULMFiT (tema 1) y modificar pocos parámetros, como hacen los métodos del tema 4 y LoRA (tema 5). El módulo 21 penalizará la divergencia de Kullback–Leibler entre el modelo ajustado y uno de referencia. Esa penalización actúa en el espacio de las distribuciones de salida, pero localmente coincide con un muelle en el espacio de parámetros cuya rigidez es la información de Fisher.

Proposición 3.11 (Desarrollo local de la divergencia KL). *Bajo las hipótesis de la proposición 3.8, y si* $\ln p_\theta$ *es tres veces derivable,*

$$D_{\mathrm{KL}}\big(p_{\theta_0} \,\big\|\, p_{\theta_0+\delta}\big) = \tfrac{1}{2}\,\delta^\top \mathcal{I}(\theta_0)\,\delta + O(\|\delta\|^3).$$

Demostración. Sea $k(\delta) = D_{\mathrm{KL}}(p_{\theta_0}\|p_{\theta_0+\delta}) = \mathbb{E}_{x\sim p_{\theta_0}}[\ln p_{\theta_0}(x) - \ln p_{\theta_0+\delta}(x)]$. Se tiene $k(0) = 0$. Su gradiente en $\delta = 0$ es $-\mathbb{E}_{p_{\theta_0}}[\nabla \ln p_{\theta_0}] = 0$ por la proposición 3.8, y su hessiano en $\delta = 0$ es $-\mathbb{E}_{p_{\theta_0}}[\nabla^2 \ln p_{\theta_0}] = \mathcal{I}(\theta_0)$ por la misma proposición. El desarrollo de Taylor de segundo orden con resto de orden tres da el enunciado. ∎

Lecturas recomendadas para este tema

- McCloskey y Cohen [24], descripción original de la interferencia catastrófica.
- Kirkpatrick *et al.* [20], sección sobre la consolidación elástica de pesos y su justificación bayesiana.
- Bishop [3], §4.4 «The Laplace Approximation».
- Goodfellow, Bengio y Courville [14], §7.1.1 «L^2 Parameter Regularization».

Ejercicios del tema 3

Ejercicio 3.1 ⋆ Enumere cuatro remedios contra el olvido catastrófico y clasifíquelos según actúen sobre los datos, sobre la pérdida, sobre el optimizador o sobre los parámetros que se entrenan. ¿Cuál de ellos permite recuperar *exactamente* el modelo base después del ajuste, y por qué?

Ejercicio 3.2 ⋆⋆⋆ En el modelo cuadrático con $\theta_0 = 1$, tarea $(\theta - 3)^2$ y muelle $\lambda(\theta - \theta_0)^2$, deduzca $\theta^*(\lambda)$ sin usar la proposición 3.3 y calcule θ^*, el olvido $(\theta^* - 1)^2$ y el error en la tarea para $\lambda = 2$. ¿Qué ocurre cuando $\lambda \to \infty$?

Ejercicio 3.3 ⋆⋆ Un modelo tiene dos parámetros, con $\theta_A = (1, 1)$ tras la tarea antigua y diagonal de Fisher $F = (4;\, 0{,}25)$. La tarea nueva tiene pérdida $\mathcal{L}_B(\theta) = (\theta_1 - 3)^2 + (\theta_2 - 3)^2$. Con $\lambda = 2$, minimice $\mathcal{L}_{\mathrm{EWC}}$ y calcule la penalización $\frac{1}{2}\sum_i F_i(\theta_i^* - 1)^2$, que aproxima el aumento de la pérdida antigua, y el error en la tarea nueva $\mathcal{L}_B(\theta^*)$. ¿Qué parámetro se mueve más y por qué?

Ejercicio 3.4 ⋆⋆ Con un solo parámetro, $\mathcal{L}_B(\theta) = (\theta - 3)^2$ y $\mathcal{L}_A(\theta) = (\theta - 1)^2$, la repetición con proporción β minimiza $(1 - \beta)\mathcal{L}_B + \beta\mathcal{L}_A$. (a) Halle $\theta^*(\beta)$ y evalúelo para $\beta = 0{,}25$. (b) Compruebe que coincide con el muelle de la proposición 3.3 para $\lambda = \beta/(1 - \beta)$.

Ejercicio 3.5 ⋆⋆ Sea una variable de Bernoulli parametrizada por su logit, $p_z(1) = \sigma(z)$ y $p_z(0) = 1 - \sigma(z)$, con σ la función logística. (a) Calcule la función de puntuación y demuestre que $\mathcal{I}(z) = \sigma(z)(1 - \sigma(z))$. (b) Evalúela en $z = 0$ y en $z = \ln 9$. (c) Si estos fueran dos parámetros de un modelo, ¿cuál quedaría más rígido con EWC?

Ejercicio 3.6 ⋆⋆⋆ Sea $\mathcal{L}_B(\theta) = \frac{1}{2}(\theta - \theta_B)^\top H(\theta - \theta_B)$, con H simétrica definida positiva de autovalores $h_1, \ldots, h_D$ y autovectores ortonormales $q_1, \ldots, q_D$, y sea la penalización $\frac{\lambda}{2}\|\theta - \theta_0\|^2$. (a) Demuestre que el minimizador es $\theta^* = (H + \lambda I)^{-1}(H\theta_B + \lambda\theta_0)$. (b) Demuestre que la

componente de $\theta^* - \theta_0$ en la dirección q_i es la de $\theta_B - \theta_0$ multiplicada por $h_i/(h_i + \lambda)$. (c) Evalúe θ^* con $H = \mathrm{diag}(10;\, 0{,}1)$, $\lambda = 1$, $\theta_0 = 0$ y $\theta_B = (1, 1)$, e interprete el resultado.

Fuente: adaptado de Goodfellow, Bengio y Courville [14], §7.1.1 (análisis cuadrático de la penalización L^2, aquí centrada en θ_0 en lugar de en el origen).

Tema 4

Ajuste eficiente en parámetros

El ajuste completo de un modelo de D parámetros produce, para cada tarea, una copia de D parámetros y exige guardar durante el entrenamiento el estado del optimizador de todos ellos. Los métodos de *ajuste eficiente en parámetros* (PEFT, del inglés *parameter-efficient fine-tuning*) congelan el modelo preentrenado y entrenan un número pequeño de parámetros nuevos. Este tema presenta los adaptadores, el ajuste de *prompt* y de prefijo, y la noción de dimensión intrínseca que explica por qué bastan tan pocos parámetros. Se sigue el §8.2 de Jurafsky y Martin [19], la clase 9 de CS224N [5] y los artículos originales [16, 21, 23, 22, 1]; LoRA, el método más usado, ocupa el tema 5.

4.1 Planteamiento

Definición 4.1 (Ajuste eficiente en parámetros). Sea $\theta_0 \in \mathbb{R}^D$ un modelo preentrenado. Un método PEFT define una familia de modelos $f_{\theta_0,\varphi}$ con parámetros nuevos $\varphi \in \mathbb{R}^P$, $P \ll D$, tal que existe φ_0 con $f_{\theta_0,\varphi_0} = f_{\theta_0}$, y entrena solo φ, partiendo de φ_0 o de un valor próximo, con θ_0 congelado.

Los métodos se agrupan en tres familias: *aditivos*, que añaden módulos o vectores nuevos (adaptadores, ajuste de *prompt* y de prefijo); *selectivos*, que entrenan un subconjunto de los parámetros existentes (la sonda lineal del tema 1 es el caso extremo), y de *reparametrización*, que escriben la actualización $\Delta\theta$ como función de pocos parámetros (subespacios aleatorios, apartado 4.4, y LoRA, tema 5).

Proposición 4.2 (Memoria del entrenamiento). *Con precisión mixta y Adam, el ajuste completo requiere unos* $16D$ *bytes para pesos, gradientes y estado del optimizador (módulo 17). Con un método PEFT y el modelo base en bf16, la memoria es de unos* $2D + 16P$ *bytes, sin contar las activaciones.*

Demostración. En el ajuste completo, cada parámetro ocupa 2 bytes de peso en bf16, 2 de gradiente, 4 de copia maestra en fp32 y 8 de los dos momentos de Adam en fp32: 16 bytes. En un método PEFT los D parámetros congelados solo necesitan su peso en bf16 (2 bytes): no tienen gradiente ni estado de Adam. Los P parámetros entrenables necesitan los 16 bytes completos. ∎

Con $D = 7 \cdot 10^9$ y $P = 2 \cdot 10^7$, la memoria baja de 112 GB a $14 + 0{,}32 = 14{,}32$ GB. Además, cada tarea se almacena con P parámetros en lugar de D, y un mismo modelo base puede servir muchas tareas cambiando solo φ.

4.2 Adaptadores

Definición 4.3 (Adaptador). Sea $h \in \mathbb{R}^d$ la salida de una subcapa de un Transformer. Un *adaptador* con dimensión de cuello $m \ll d$ calcula

$$a(h) = h + W_\uparrow \, g\big(W_\downarrow h + b_\downarrow\big) + b_\uparrow, \qquad W_\downarrow \in \mathbb{R}^{m\times d},\ W_\uparrow \in \mathbb{R}^{d\times m},$$

con g una no linealidad. Houlsby *et al.* [16] insertan dos adaptadores por capa, tras la subcapa de atención y tras la red de alimentación hacia delante, y entrenan además los parámetros de normalización de capa.

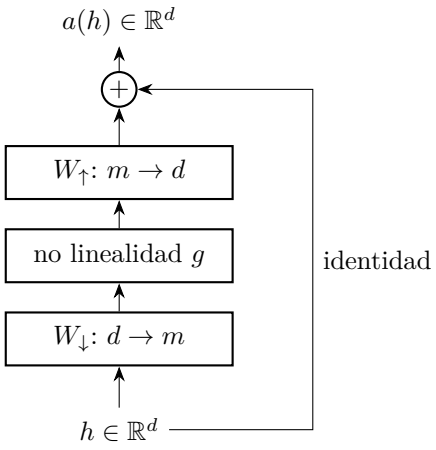


Figura 4.1: Adaptador con cuello de botella. Si $W_\uparrow$ y $b_\uparrow$ son nulos, el adaptador es la identidad.

Proposición 4.4. *Un adaptador tiene $2md + m + d$ parámetros. Si se inicializa con $W_\uparrow = 0$ y $b_\uparrow = 0$, entonces $a(h) = h$ para todo h, de modo que el modelo con adaptadores coincide al principio con el preentrenado.*

Demostración. $W_\downarrow$ y $W_\uparrow$ aportan md parámetros cada una; $b_\downarrow \in \mathbb{R}^m$ y $b_\uparrow \in \mathbb{R}^d$, $m + d$. Con $W_\uparrow = 0$ y $b_\uparrow = 0$, el sumando $W_\uparrow g(\cdot) + b_\uparrow$ es nulo. ■

Houlsby *et al.* usan una inicialización *casi* nula, de modo que el adaptador es casi la identidad, y obtienen en GLUE resultados a menos de un 0,4 % de los del ajuste completo añadiendo un 3,6 % de parámetros por tarea [16].

Observación 4.5 (Latencia). Los adaptadores añaden capas que se ejecutan en serie y no pueden fusionarse con los pesos originales, porque contienen una no linealidad. Hu *et al.* [18, §3] midieron aumentos de la latencia de inferencia superiores al 30 % con secuencias cortas y lotes pequeños. LoRA evita este problema (tema 5).

4.3 Ajuste de *prompt* y de prefijo

Definición 4.6 (Ajuste de *prompt*). Sea $E \in \mathbb{R}^{T\times d}$ la matriz de *embeddings* de una entrada. El *ajuste de prompt* antepone p vectores entrenables $P_e \in \mathbb{R}^{p\times d}$ y alimenta el modelo congelado con $[P_e; E] \in \mathbb{R}^{(p+T)\times d}$. Tiene pd parámetros [21].

Los vectores de P_e no tienen por qué corresponder a ningún token del vocabulario: forman un *prompt continuo* (*soft prompt*). Lester *et al.* [21] observaron que la diferencia con el ajuste completo disminuye con la escala y prácticamente desaparece en modelos de unos 10^{10} parámetros.

Definición 4.7 (Ajuste de prefijo). El *ajuste de prefijo* antepone, en cada una de las L capas, p claves y p valores entrenables, $P_K^l, P_V^l \in \mathbb{R}^{p\times d}$, a las claves y valores calculados a partir de la entrada. Tiene $2Lpd$ parámetros. Para estabilizar el entrenamiento, Li y Liang [23] reparametrizan los prefijos mediante una pequeña red que se descarta al terminar; con ello entrenan en torno al 0,1 % de los parámetros.

El efecto de un prefijo sobre la atención admite una descripción exacta.

Teorema 4.8 (El prefijo interpola la atención). *Sean $q \in \mathbb{R}^d$ una consulta, $K, V \in \mathbb{R}^{T\times d}$ las claves y valores de la entrada y $P_K, P_V \in \mathbb{R}^{p\times d}$ los del prefijo. Escríbase* $\mathrm{At}(q,K,V) = \mathrm{softmax}(Kq/\sqrt{d})^\top V$. *Entonces*

$$\mathrm{At}\big(q,[P_K;K],[P_V;V]\big) = \big(1-\mu(q)\big)\,\mathrm{At}(q,K,V) + \mu(q)\,\mathrm{At}(q,P_K,P_V),$$

con

$$\mu(q) = \frac{S_P}{S_P+S_K}, \qquad S_P = \sum_{j=1}^{p} e^{(P_K)_j\cdot q/\sqrt{d}}, \quad S_K = \sum_{i=1}^{T} e^{k_i\cdot q/\sqrt{d}}.$$

Demostración. Sean $a_i = e^{k_i\cdot q/\sqrt{d}}$ y $b_j = e^{(P_K)_j\cdot q/\sqrt{d}}$. La atención sobre la concatenación es

$$\frac{\sum_j b_j (P_V)_j + \sum_i a_i v_i}{S_P+S_K} = \frac{S_P}{S_P+S_K}\cdot\frac{\sum_j b_j (P_V)_j}{S_P} + \frac{S_K}{S_P+S_K}\cdot\frac{\sum_i a_i v_i}{S_K}.$$

Los dos cocientes de la derecha son, por definición de la softmax, $\mathrm{At}(q,P_K,P_V)$ y $\mathrm{At}(q,K,V)$, y $S_K/(S_P+S_K) = 1-\mu(q)$. ∎

Corolario 4.9. *El ajuste de prefijo no altera los pesos relativos de atención entre las posiciones de la entrada: solo mezcla la salida original con un vector que depende de la consulta, con una proporción $\mu(q) \in (0,1)$.*

Esta lectura, que relaciona el ajuste de prefijo con los adaptadores, se desarrolla en He *et al.* [15].

4.4 Dimensión intrínseca

¿Por qué basta con ajustar una fracción mínima de los parámetros? Li *et al.* [22] propusieron medir la dimensión efectiva de un problema de optimización restringiendo el entrenamiento a un subespacio aleatorio.

Definición 4.10 (Subespacio aleatorio y dimensión intrínseca). Sean $\theta_0 \in \mathbb{R}^D$ y $P \in \mathbb{R}^{D\times d'}$ una matriz aleatoria fija. Se entrena solo $z \in \mathbb{R}^{d'}$, inicializado en 0, con la reparametrización $\theta = \theta_0 + Pz$. La *dimensión intrínseca* d_{90} de una tarea es el menor d' con el que se alcanza el 90 % del rendimiento del ajuste completo [1, §3].

Proposición 4.11 (Gradiente en el subespacio). *Si $\mathcal{L}$ es derivable, $\nabla_z \mathcal{L}(\theta_0+Pz) = P^\top \nabla_\theta \mathcal{L}(\theta)\big|_{\theta=\theta_0+Pz}$.*

Demostración. La aplicación $z \mapsto \theta_0 + Pz$ es afín con jacobiana P. Por la regla de la cadena (módulo 4), $\nabla_z \mathcal{L} = P^\top \nabla_\theta \mathcal{L}$. ■

Aghajanyan *et al.* [1] estimaron la dimensión intrínseca del ajuste de modelos de lenguaje preentrenados, usando para P la transformada Fastfood, que evita almacenar una matriz densa de $D \times d'$. Sus tres resultados principales son:

(i) con RoBERTa-Large, unos 200 parámetros bastan para alcanzar el 90 % del rendimiento del ajuste completo en la tarea MRPC [1, §4.2];

(ii) la dimensión intrínseca disminuye a lo largo del preentrenamiento y es menor en los modelos con más parámetros [1, §5.1 y §5.2];

(iii) una cota de generalización basada en compresión depende de la dimensión intrínseca y no del número total de parámetros: con n ejemplos, el error de generalización es $O(\sqrt{d_{90}/n})$, para modelos entrenados en el subespacio [1, §5.3].

Observación 4.12. El segundo resultado explica en parte la paradoja de que los modelos mayores se ajusten con menos datos: el preentrenamiento comprime el problema de adaptación. Hu *et al.* formularon a partir de aquí la hipótesis que motiva LoRA [18, §1 y §4.1]: si el ajuste vive en un subespacio de dimensión pequeña, también la actualización de cada matriz de pesos debe tener un *rango intrínseco* bajo. LoRA no fija las direcciones al azar: aprende la mejor base de rango r.

Lecturas recomendadas para este tema

- Jurafsky y Martin [19], §8.2 «Parameter Efficient Fine Tuning».
- CS224N [5], clase 9 «Efficient Adaptation (Prompting + PEFT)».
- Houlsby *et al.* [16], sección sobre los adaptadores y su inicialización.
- Lester *et al.* [21] y Li y Liang [23].
- Aghajanyan *et al.* [1], §3 (definición), §4 (resultados) y §5 (relación con el preentrenamiento y la generalización).

Ejercicios del tema 4

Ejercicio 4.1 ⋆ Sobre un cuerpo preentrenado con representaciones de dimensión $h = 768$ y unos $1{,}1 \cdot 10^8$ parámetros se coloca una cabeza lineal de 2 clases con sesgo. Se congelan todos los parámetros anteriores a la cabeza. (a) ¿Cuántos parámetros se entrenan y qué fracción del total representan? (b) ¿A qué familia de métodos PEFT pertenece este procedimiento? (c) ¿Cabe esperar más o menos exactitud que con el ajuste completo? Razone en qué casos la diferencia sería pequeña.

Fuente: adaptado de Zhang *et al.* [28], §14.2, ejercicio 3.

Ejercicio 4.2 ⋆⋆ Un Transformer de 12 capas con $d = 768$ y unos $1{,}1 \cdot 10^8$ parámetros lleva dos adaptadores por capa con $m = 64$ (se ignoran los parámetros de normalización). (a) Calcule los parámetros de los adaptadores y su porcentaje respecto al modelo. (b) Compare el almacenamiento necesario para 20 tareas con adaptadores y con 20 copias ajustadas por completo.

Ejercicio 4.3 ⋆⋆ Un modelo tiene $L = 24$ capas y $d = 1024$. Calcule los parámetros entrenables del ajuste de *prompt* y del ajuste de prefijo con $p = 20$ vectores (sin contar la red de reparametrización) y el cociente entre ambos.

Ejercicio 4.4 ⋆⋆ Sean $D = 3$, $\theta_0 = (1, 0, 2)$, $P = (1, 2, -1)^\top$ (de modo que $d' = 1$) y la pérdida $\mathcal{L}(\theta) = \|\theta - \theta^\star\|^2$ con $\theta^\star = (2, 3, 1)$. (a) Halle el z óptimo y el θ correspondiente. (b) Calcule la pérdida que queda y compárela con la del ajuste completo. (c) Compruebe la proposición 4.11 en $z = 0$.

Ejercicio 4.5 ⋆⋆⋆ En la reparametrización $\theta = \theta_0 + Pz$: (a) demuestre que el conjunto de modelos alcanzables es el subespacio afín $\theta_0 + \mathrm{Col}(P)$, de dimensión rango P; (b) demuestre que un paso de descenso de gradiente en z con tasa η desplaza θ en $-\eta P P^\top \nabla_\theta \mathcal{L}$; (c) si las columnas de P son ortonormales, demuestre que ese desplazamiento es la proyección ortogonal sobre $\mathrm{Col}(P)$ del paso de descenso de gradiente del ajuste completo.

Tema 5

LoRA: adaptación de bajo rango

LoRA (*low-rank adaptation*) congela cada matriz preentrenada y aprende solo una corrección de rango bajo, escrita como producto de dos matrices delgadas. Es hoy el método PEFT más usado porque no añade latencia de inferencia y su análisis se reduce al álgebra lineal del módulo 3. Este tema sigue el artículo original de Hu *et al.* [18, §4 y §7], el teorema de Eckart y Young [12] en la forma del módulo 3 y de Deisenroth, Faisal y Ong [10, §4.6], y la clase invitada sobre LoRA del curso CS224N [5].

5.1 Definición y número de parámetros

Definición 5.1 (LoRA). Sea $W \in \mathbb{R}^{d\times k}$ una matriz preentrenada de una capa lineal $h = Wx$, con $x \in \mathbb{R}^k$. Sean $r \ll \min(d,k)$ el *rango*, $\alpha > 0$ un hiperparámetro de escala y $s = \alpha/r$. LoRA sustituye la capa por

$$h = Wx + s\,BAx, \qquad A \in \mathbb{R}^{r\times k},\ B \in \mathbb{R}^{d\times r},$$

con W congelada y A, B entrenables. Se inicializa A con entradas gaussianas y $B = 0$ [18, §4.1]. La matriz efectiva es $W' = W + s\,BA$.

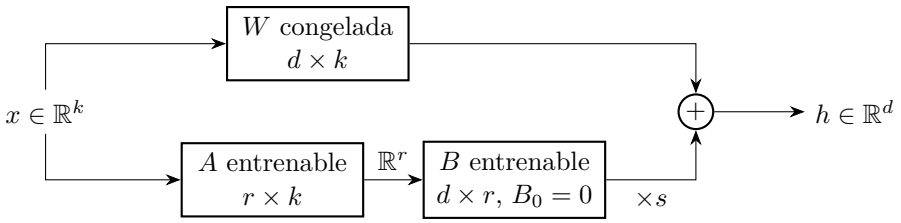


Figura 5.1: Capa lineal con LoRA. La entrada atraviesa la matriz congelada y, en paralelo, el cuello de dimensión r formado por A y B. Como B empieza en cero, al principio $h = Wx$.

Proposición 5.2 (Parámetros y coste). *LoRA entrena $r(d+k)$ parámetros por matriz, una fracción $r(d+k)/(dk) = r(1/d + 1/k)$ de los dk del ajuste completo. Calcular $s\,B(Ax)$ cuesta $2r(d+k)$ operaciones más un escalado, frente a las $2dk$ de Wx, y nunca requiere formar BA.*

Demostración. A tiene rk entradas y B, dr. El producto Ax es $(r\times k)(k\times 1)$ y cuesta $2rk$ operaciones (módulo 17, tema 1); $B(Ax)$ es $(d\times r)(r\times 1)$ y cuesta $2dr$. ∎

Ejemplo 5.3 (Una capa LoRA a mano). Sean $W = I_2$, $B = (1,2)^\top$, $A = (0{,}5;\ -1)$, $r = 1$ y $\alpha = 2$, de modo que $s = 2$. Entonces

$$BA = \begin{pmatrix} 0{,}5 & -1 \\ 1 & -2 \end{pmatrix}, \qquad W' = I + 2BA = \begin{pmatrix} 2 & -2 \\ 2 & -3 \end{pmatrix}.$$

Cuadro 5.1: Parámetros entrenables de LoRA en una matriz.

Matriz	r	Completo dk	LoRA $r(d+k)$	Fracción
64×64	4	4096	512	12,5 %
768×768	4	589 824	6144	1,04 %
4096×4096	8	16 777 216	65 536	0,39 %
4096×4096	16	16 777 216	131 072	0,78 %

Para $x = (1, 0)$: $Ax = 0{,}5$, $B(Ax) = (0{,}5;\ 1)$ y $h = (1, 0) + 2 \cdot (0{,}5;\ 1) = (2, 2)$, que coincide con $W'x$.

En el mini-GPT de la práctica ($d = 64$, red de alimentación de dimensión 256, 2 bloques), LoRA con $r = 4$ en las cuatro matrices de atención (64×64) y las dos de la red de alimentación (64×256 y 256×64) de cada bloque entrena $2 \cdot (4 \cdot 512 + 2 \cdot 1280) = 9216$ parámetros, el 8,57 % de los 107 520 del modelo. En GPT-3 de 175 000 millones de parámetros, Hu *et al.* redujeron los parámetros entrenables unas 10 000 veces y la memoria de GPU unas 3 veces respecto al ajuste completo con Adam; con $r = 4$ en W_Q y W_V, el punto de control pasó de unos 350 GB a unos 35 MB [18, §4.2].

Proposición 5.4 (Fusión). *Tras el entrenamiento, sustituir W por $W' = W + s\,BA$ produce la misma función que la capa LoRA y el mismo coste de inferencia que la capa original.*

Demostración. Por la distributividad del producto de matrices, $W'x = Wx + s\,(BA)x = Wx + s\,B(Ax)$ para todo x. W' tiene la misma forma $d \times k$ que W, luego $W'x$ cuesta $2dk$ operaciones, como Wx. ∎

Así, LoRA no añade latencia de inferencia, a diferencia de los adaptadores del tema 4. Sin fusionar, en cambio, un mismo modelo base puede servir muchos adaptadores pequeños e intercambiarlos en cada petición.

5.2 Rango de la corrección

Proposición 5.5 (Cota de rango). *Para todo $A \in \mathbb{R}^{r \times k}$ y $B \in \mathbb{R}^{d \times r}$,* $\operatorname{rango}(BA) \le r$.

Demostración. Para todo $x \in \mathbb{R}^k$, $(BA)x = B(Ax)$ es una combinación lineal de las r columnas de B con coeficientes las componentes de $Ax \in \mathbb{R}^r$. Luego $\operatorname{im}(BA) \subseteq \operatorname{im}(B) = \operatorname{Col}(B)$, que está generado por r vectores y tiene dimensión a lo sumo r. El rango es la dimensión de la imagen. ∎

Proposición 5.6 (Recíproco). *Toda matriz $M \in \mathbb{R}^{d \times k}$ con* $\operatorname{rango} M \le r$ *puede escribirse como $M = BA$ con $B \in \mathbb{R}^{d \times r}$ y $A \in \mathbb{R}^{r \times k}$.*

Demostración. Sea $M = \sum_{i=1}^{\rho} \sigma_i u_i v_i^\top$ la SVD reducida de M, con $\rho = \operatorname{rango} M \le r$ (módulo 3). Tómense como columnas de B los vectores $\sigma_1 u_1, \ldots, \sigma_\rho u_\rho$, completados con $r - \rho$ columnas nulas, y como filas de A los vectores $v_1^\top, \ldots, v_\rho^\top$, completados con $r - \rho$ filas nulas. Entonces $BA = \sum_{i=1}^{\rho} \sigma_i u_i v_i^\top = M$. ∎

Corolario 5.7. *El conjunto de correcciones que LoRA de rango r puede representar es exactamente $\{\Delta W \in \mathbb{R}^{d \times k} : \operatorname{rango} \Delta W \le r\}$. Si $r < \min(d, k)$, ninguna corrección de rango completo es representable.*

5.3 Gradientes e inicialización

Proposición 5.8 (Gradientes de LoRA). *Sea $h = Wx + s\,BAx$ y sea $G = \partial\mathcal{L}/\partial h \in \mathbb{R}^d$ el gradiente que llega a la salida. Entonces*

$$\frac{\partial \mathcal{L}}{\partial B} = s\,G\,(Ax)^\top \in \mathbb{R}^{d\times r}, \qquad \frac{\partial \mathcal{L}}{\partial A} = s\,B^\top G\,x^\top \in \mathbb{R}^{r\times k}.$$

Con un lote, se suman las contribuciones de cada ejemplo.

Demostración. Se usa la regla del módulo 4: si $h = Mz$ con z fijo, entonces $\partial\mathcal{L}/\partial M = Gz^\top$. (i) Respecto a B, h depende de B a través de $s\,B(Ax)$, que es de la forma Mz con $M = B$ y $z = s\,Ax$; luego $\partial\mathcal{L}/\partial B = G\,(s\,Ax)^\top$. (ii) Respecto a A, sea $c = Ax \in \mathbb{R}^r$. Como h depende de c a través de $s\,Bc$, se tiene $\partial\mathcal{L}/\partial c = s\,B^\top G$. Y como $c = Ax$ es de la forma Mz con $M = A$ y $z = x$, $\partial\mathcal{L}/\partial A = (s\,B^\top G)\,x^\top$. ■

Corolario 5.9 (Inicialización). *Con $B = 0$ y A aleatoria:*

(i) $BA = 0$ y el modelo ajustado empieza siendo exactamente el modelo base;

(ii) en el primer paso $\partial\mathcal{L}/\partial A = 0$, pero $\partial\mathcal{L}/\partial B = s\,G(Ax)^\top \neq 0$ si $G \neq 0$ y $Ax \neq 0$, de modo que B deja de ser nula y, desde el segundo paso, también A recibe gradiente.

En cambio, $(A, B) = (0, 0)$ es un punto fijo del descenso de gradiente (ejercicio 5.7).

La elección simétrica, $A = 0$ con B aleatoria, también empieza en el modelo base y también permite aprender; el convenio del artículo original es $B = 0$.

5.4 La escala α/r

Proposición 5.10 (Adam es invariante a la escala del gradiente). *Sea el optimizador Adam con $\varepsilon = 0$ (módulo 7). Si se multiplican todos los gradientes por una constante $c > 0$, la sucesión de actualizaciones de los parámetros no cambia.*

Demostración. Con gradientes cg_t, los momentos pasan a ser $m'_t = c\,m_t$ y $v'_t = c^2 v_t$, por ser medias móviles exponenciales lineales en g_t y en g_t^2 respectivamente; las correcciones de sesgo no dependen de c. La actualización es $\eta\,\hat{m}'_t/\sqrt{\hat{v}'_t} = \eta\,c\,\hat{m}_t/(c\sqrt{\hat{v}_t}) = \eta\,\hat{m}_t/\sqrt{\hat{v}_t}$. ■

Por la proposición 5.8, el gradiente de B es proporcional a s. Por la proposición 5.10, la actualización ΔB del primer paso no depende de s, y la variación de la función, $s\,\Delta B\,A\,x$, es proporcional a s. El factor $s = \alpha/r$ actúa, por tanto, como un multiplicador de la tasa de aprendizaje. Hu *et al.* observan que ajustar α con Adam equivale aproximadamente a ajustar la tasa, fijan α al primer valor de r que prueban y no lo modifican; dividir por r reduce la necesidad de reajustar hiperparámetros al cambiar el rango [18, §4.1]. En la práctica se fija α (por ejemplo, $\alpha = 16$ o $\alpha = 2r$) y se ajusta r.

5.5 Por qué funciona: relación con la SVD

LoRA apuesta a que la actualización ΔW que necesita el ajuste fino tiene un espectro que decae deprisa. El teorema de Eckart–Young (módulo 3, tema 5) cuantifica lo que se pierde al restringir el rango.

Teorema 5.11 (Eckart–Young, norma de Frobenius). *Sea $\Delta W = \sum_i \sigma_i u_i v_i^\top$ la SVD, con $\sigma_1 \geq \sigma_2 \geq \cdots$. Para todo r, la matriz $\Delta W_r = \sum_{i \leq r} \sigma_i u_i v_i^\top$ minimiza $\|\Delta W - M\|_F$ entre las matrices de rango a lo sumo r, y*

$$\|\Delta W - \Delta W_r\|_F = \Big(\sum_{i>r} \sigma_i^2\Big)^{1/2}.$$

La demostración está en el módulo 3 y en [12].

Corolario 5.12 (Error mínimo de LoRA). *Sea $\Delta W^\star$ la actualización ideal de una matriz. Entonces*

$$\min_{A,B} \|s\,BA - \Delta W^\star\|_F = \Big(\sum_{i>r} \sigma_i^2(\Delta W^\star)\Big)^{1/2}.$$

Demostración. Como $s > 0$, $\{s\,BA\}$ recorre el mismo conjunto que $\{BA\}$, que por el corolario 5.7 es el de las matrices de rango a lo sumo r. Basta aplicar el teorema 5.11. ■

Ejemplo 5.13. Si los valores singulares de $\Delta W^\star$ son (3; 1; 0,1), la energía total es $\sum \sigma_i^2 = 9 + 1 + 0{,}01 = 10{,}01$. La mejor corrección de rango 1 tiene error $\sqrt{1 + 0{,}01} = 1{,}005$ y conserva $9/10{,}01 = 89{,}9\,\%$ de la energía; la de rango 2 tiene error 0,1 y conserva $10/10{,}01 = 99{,}9\,\%$.

Hay dos diferencias con truncar una SVD. LoRA no calcula ninguna SVD: el descenso de gradiente encuentra B y A directamente. Y LoRA aproxima la *actualización*, no W, que suele tener rango completo y no se puede comprimir así sin perder mucho.

Observación 5.14 (Evidencia empírica). Hu *et al.* adaptan principalmente W_Q y W_V y encuentran que, en algunas tareas, un rango tan pequeño como $r = 1$ basta [18, §7.1 y §7.2]; las direcciones singulares principales aprendidas con $r = 8$ y con $r = 64$ se solapan mucho, y el resto parecen ruido. Dettmers *et al.* observaron, en cambio, que para igualar el ajuste completo en modelos grandes hace falta aplicar LoRA a todas las capas lineales [11, §4]. En la práctica del módulo, la actualización del ajuste completo en W_Q tiene rango 64 y sus 4 primeros valores singulares solo concentran el 41,5 % de la energía: un modelo tan pequeño, ajustado de forma tan agresiva, no cumple bien la hipótesis. Aun así, LoRA acierta más en la prueba (0,450 frente a 0,300, con tasas de aprendizaje distintas); una explicación plausible es que restringir el rango regulariza.

Lecturas recomendadas para este tema

- Hu *et al.* [18], §4.1 (formulación, inicialización y escala), §4.2 (aplicación a los Transformer) y §7 (análisis del rango).
- Módulo 3, tema 5, y Deisenroth, Faisal y Ong [10], §4.6 «Matrix Approximation».
- CS224N [5], clase invitada «Tinker and LoRA Without Regret».
- Jurafsky y Martin [19], §8.2.

Ejercicios del tema 5

Ejercicio 5.1 ⋆ Justifique si son verdaderas o falsas: (a) LoRA modifica los pesos W durante el entrenamiento; (b) si se inicializan A y B a cero, LoRA funciona igual; (c) tras fusionar un adaptador, la inferencia es igual de rápida que con el modelo base.

Ejercicio 5.2 ⋆⋆ (a) Una matriz de la red de alimentación mide 1024×4096. ¿Cuántos parámetros tiene? ¿Cuántos entrena LoRA con $r = 8$ y qué porcentaje es? (b) Un modelo de 12 capas con $d = 768$ aplica LoRA con $r = 8$ a W_Q, W_K, W_V y W_O, todas de 768×768. Calcule los parámetros entrenables, los de esas matrices y el porcentaje.

Ejercicio 5.3 ⋆⋆ Sean $W = I_2$, $B = (1, 0)^\top$, $A = (2, 1)$, $r = 1$ y $\alpha = 2$. Calcule la matriz fusionada W' y $W'x$ para $x = (1, 1)$, y compruebe el resultado por el camino sin fusionar.

Ejercicio 5.4 ⋆⋆ En una capa LoRA con $s = 2$, $B = (0, 0)^\top$ y $A = (1, 1)$ llega el gradiente $G = (1, -1)$ para la entrada $x = (2, 1)$. Calcule $\partial\mathcal{L}/\partial B$ y $\partial\mathcal{L}/\partial A$.

Ejercicio 5.5 ⋆⋆ Sea $\Delta W = \begin{pmatrix} 2 & 1 & 1 \\ 1 & 2 & -1 \end{pmatrix}$. Calcule su SVD a partir de $\Delta W\, \Delta W^\top$, su mejor aproximación de rango 1, el error de Frobenius y la energía conservada. Escriba esa aproximación como un producto BA con $B \in \mathbb{R}^{2\times 1}$ y $A \in \mathbb{R}^{1\times 3}$.

Fuente: adaptado de Deisenroth, Faisal y Ong [10], cap. 4, ejercicio 4.10.

Ejercicio 5.6 ⋆⋆⋆ Demuestre que $\operatorname{rango}(BA) \le r$ y que, por tanto, si $r < \min(d, k)$ existen actualizaciones ΔW que LoRA no puede representar exactamente. Dé un ejemplo con $d = k = 2$ y $r = 1$ y compruébelo con el determinante.

Ejercicio 5.7 ⋆⋆⋆ Use la proposición 5.8 para demostrar que, con $A = 0$ y $B = 0$, los parámetros no cambian nunca con descenso de gradiente (ni con Adam). Demuestre después que, con $B = 0$ y $A \neq 0$, la actualización de B en el primer paso es distinta de cero si $G \neq 0$ y $Ax \neq 0$.

Tema 6

QLoRA y elección de estrategia

Con LoRA, la matriz W no recibe actualizaciones: solo se lee. Puede guardarse, por tanto, en muy baja precisión. QLoRA combina la cuantización del modelo base a 4 bits con adaptadores LoRA entrenados en 16 bits y permite ajustar modelos de decenas de miles de millones de parámetros en una sola GPU. Este tema estudia sus tres ingredientes (el formato NF4, la doble cuantización y los optimizadores paginados), hace la contabilidad de memoria de cada configuración y termina con los criterios para elegir entre ajuste fino, instrucciones en el contexto y recuperación de documentos. Se sigue el artículo de Dettmers *et al.* [11, §3 y §4], la contabilidad de parámetros de la tarea 1 de CS336 [7, §3.5] y el capítulo 8 de Jurafsky y Martin [19]; la cuantización por bloques se estudió en el módulo 19.

6.1 Cuantización por bloques

Definición 6.1 (Cuantización por bloques con un libro de códigos). Sea $\mathcal{Q} = \{q_0 < q_1 < \cdots < q_{2^b-1}\} \subset [-1, 1]$ un *libro de códigos* de 2^b niveles. Los pesos se dividen en bloques de B elementos. Para cada bloque $w \in \mathbb{R}^B$ se guarda la *constante de escala* $c = \max_j |w_j|$ y, para cada j, el índice del nivel de $\mathcal{Q}$ más próximo a w_j/c. La *descuantización* devuelve $\hat{w}_j = c\, q_{i_j}$.

Proposición 6.2 (Bits por parámetro). *Si cada índice ocupa b bits y cada constante se guarda en coma flotante de 32 bits, la cuantización por bloques ocupa $b + 32/B$ bits por parámetro.*

Demostración. Un bloque de B pesos ocupa Bb bits de índices y 32 bits de constante: $(Bb + 32)/B = b + 32/B$ bits por peso. ∎

Con $b = 4$ y $B = 64$, el coste es $4 + 0{,}5 = 4{,}5$ bits por parámetro: la constante de escala añade un 12,5 % al tamaño de los índices.

6.2 El formato NF4

La elección del libro de códigos importa. Los pesos de una red preentrenada siguen aproximadamente una distribución normal de media nula, de modo que los valores próximos a cero son mucho más frecuentes que los extremos. Un libro de códigos uniforme desperdicia niveles en zonas poco pobladas.

Proposición 6.3 (Cuantización por cuantiles). *Sea $X \sim \mathcal{N}(0, 1)$ y sean $\beta_i = \Phi^{-1}(i/2^b)$, $i = 0, \ldots, 2^b$, con $\beta_0 = -\infty$ y $\beta_{2^b} = +\infty$. Si se asigna el código i a los valores de $[\beta_i, \beta_{i+1})$, cada código tiene probabilidad 2^{-b}, y la entropía del código es b bits, la máxima posible con 2^b símbolos.*

Demostración. Como Φ es continua y estrictamente creciente, $P(\beta_i \le X < \beta_{i+1}) = \Phi(\beta_{i+1}) - \Phi(\beta_i) = (i+1)/2^b - i/2^b = 2^{-b}$. Una distribución uniforme sobre 2^b símbolos tiene entropía $\log_2 2^b = b$ bits, que es la máxima posible (módulo 6). ■

Así, cada uno de los 2^b códigos se usa con la misma frecuencia y ninguno se desperdicia. El *NormalFloat* de 4 bits (NF4) de Dettmers *et al.* aplica esta idea con $b = 4$: estima los niveles a partir de cuantiles de la normal estándar, los normaliza al intervalo $[-1, 1]$ y, para que el cero sea representable exactamente, construye por separado 2^{b-1} niveles negativos y $2^{b-1} + 1$ positivos y elimina el cero repetido [11, §3]. El tamaño de bloque es $B = 64$.

Ejemplo 6.4 (Una versión simplificada con 2 bits). Tómense como niveles los puntos medios en probabilidad de cada intervalo, $\Phi^{-1}((i + \frac{1}{2})/2^b)$, normalizados por el mayor. Con $b = 2$: $\Phi^{-1}(1/8) = -1{,}1503$, $\Phi^{-1}(3/8) = -0{,}3186$ y, por simetría, $\Phi^{-1}(5/8) = 0{,}3186$ y $\Phi^{-1}(7/8) = 1{,}1503$. Dividiendo por 1,1503 se obtiene $\mathcal{Q} = \{-1; -0{,}2770; 0{,}2770; 1\}$: los niveles se concentran cerca de cero. Este libro de códigos no contiene el cero; NF4 lo añade expresamente.

6.3 Doble cuantización

Proposición 6.5 (Coste de las constantes con doble cuantización). *Si las constantes de escala de primer nivel, una por bloque de B_1 pesos, se cuantizan a su vez a 8 bits en bloques de B_2 constantes, cada una con una constante de segundo nivel de 32 bits, el coste de las constantes es $8/B_1 + 32/(B_1 B_2)$ bits por parámetro.*

Demostración. Cada parámetro comparte una constante de 8 bits con otros $B_1 - 1$: $8/B_1$ bits. Cada constante de segundo nivel, de 32 bits, sirve a B_2 constantes de primer nivel y, por tanto, a $B_1 B_2$ parámetros: $32/(B_1 B_2)$ bits. ■

Con $B_1 = 64$ y $B_2 = 256$, los valores de QLoRA [11, §3], el coste pasa de $32/64 = 0{,}5$ a $8/64 + 32/16\,384 = 0{,}125 + 0{,}00195 = 0{,}127$ bits por parámetro: un ahorro de 0,373 bits por parámetro. Antes de cuantizarlas, las constantes se centran restando su media, lo que permite una cuantización simétrica.

6.4 QLoRA

Definición 6.6 (QLoRA). QLoRA almacena los pesos congelados del modelo base en NF4 con doble cuantización y añade adaptadores LoRA en bf16 a las capas lineales. La pasada hacia delante de una capa es

$$Y = X\,\mathrm{dq}(c_1, c_2, W_{\mathrm{NF4}}) + s\,X L_1 L_2,$$

donde dq descuantiza primero las constantes y después los pesos, produciendo una matriz en bf16, y L_1, L_2 son los factores LoRA [11, §3, ecs. 5 y 6]. El tipo de almacenamiento es NF4 y el tipo de cálculo, bf16. Solo se calculan gradientes respecto a los parámetros de LoRA.

En esta definición se usa el convenio de filas de [11], $Y = XW$; los factores L_1, L_2 desempeñan el papel de $A^\top$ y $B^\top$. Aunque W no recibe gradiente, la retropropagación sí necesita multiplicar por W descuantizada para propagar el gradiente hacia las capas anteriores, cuyos adaptadores también se entrenan.

El tercer ingrediente son los *optimizadores paginados*: el estado del optimizador se guarda en memoria unificada, que el controlador traslada automáticamente a la memoria de la CPU cuando la GPU se queda sin espacio y devuelve cuando hace falta para la actualización. Con estos tres elementos, Dettmers *et al.* ajustaron un modelo de 65 000 millones de parámetros en una sola GPU de 48 GB, frente a los más de 780 GB que requería el ajuste completo en 16 bits [11, §1]. Su mejor modelo, Guanaco 65B, alcanzó el 99,3 % del rendimiento de ChatGPT en la prueba de Vicuna según la evaluación de GPT-4, con un intervalo de confianza del 95 % de ±4,4 puntos. Los propios autores advierten que no demostraron que QLoRA iguale al ajuste completo en 16 bits a las escalas de 33 000 y 65 000 millones de parámetros [11, §8].

Proposición 6.7 (Memoria de pesos y optimizador). *Sin contar las activaciones, la memoria de pesos y estado del optimizador es de unos* $16N$ *bytes en el ajuste completo con Adam en precisión mixta y de unos* $(b_{\text{ef}}/8)\,N + 16N_{\text{LoRA}}$ *bytes en QLoRA, donde* b_{ef} *son los bits efectivos por parámetro del modelo base.*

Demostración. El primer caso es la proposición 4.2. En QLoRA, el modelo base ocupa b_{ef} bits, es decir, $b_{\text{ef}}/8$ bytes por parámetro, sin gradientes ni estado del optimizador; cada parámetro de LoRA necesita peso, gradiente, copia maestra y dos momentos: 16 bytes. ■

Cuadro 6.1: Memoria de pesos y optimizador de un modelo de $7 \cdot 10^9$ parámetros, sin activaciones, tomando 4 bits por parámetro para la base cuantizada ($1\,\text{GB} = 10^9$ bytes).

Configuración	Cálculo	Memoria
Pesos en bf16	$7 \cdot 10^9 \times 2$ B	14 GB
Ajuste completo con Adam, precisión mixta	$7 \cdot 10^9 \times 16$ B	112 GB
QLoRA: base en 4 bits	$7 \cdot 10^9 \times 0{,}5$ B	3,5 GB
QLoRA: $2 \cdot 10^7$ parámetros LoRA	$2 \cdot 10^7 \times 16$ B	0,32 GB
QLoRA en total	$3{,}5 + 0{,}32$	3,82 GB

La reducción es de unas $112/3{,}82 \approx 29$ veces y permite ajustar un modelo de este tamaño en una GPU de consumo. Con las constantes de escala, $b_{\text{ef}} \approx 4{,}127$ en lugar de 4, lo que añade unos 0,11 GB.

6.5 ¿Ajustar, dar instrucciones en el contexto o recuperar documentos?

El ajuste fino no es la única forma de adaptar un modelo. Las otras dos son dar instrucciones y ejemplos en el contexto (*prompting* y aprendizaje con pocos ejemplos) y recuperar documentos relevantes e insertarlos en el contexto (RAG, módulo 22). La regla práctica es que el ajuste cambia *cómo* responde el modelo y el contexto cambia *qué sabe* en cada consulta.

En la práctica, las tres formas se combinan: un modelo ajustado para seguir un formato y citar sus fuentes, alimentado con documentos recuperados. Los proveedores de modelos comerciales ofrecen públicamente, según el caso, ajuste fino a través de sus plataformas; los detalles técnicos de esos servicios no siempre se publican.

Lecturas recomendadas para este tema

Cuadro 6.2: Criterios para elegir la forma de adaptación.

Criterio	Instrucciones en el contexto	RAG	Ajuste fino
Coste inicial	ninguno	índice de documentos	datos y entrenamiento
Conocimiento nuevo o cambiante	solo si cabe en el contexto	sí, con fuentes citables	mal: caro de actualizar y propenso a inventar
Formato, estilo y tono	aceptable	no ayuda	muy bueno
Coste por consulta	contextos largos	recuperación y contexto	contextos cortos
Riesgo de olvido	ninguno	ninguno	sí (tema 3)
Acceso a los pesos	no	no	sí, o una API de ajuste

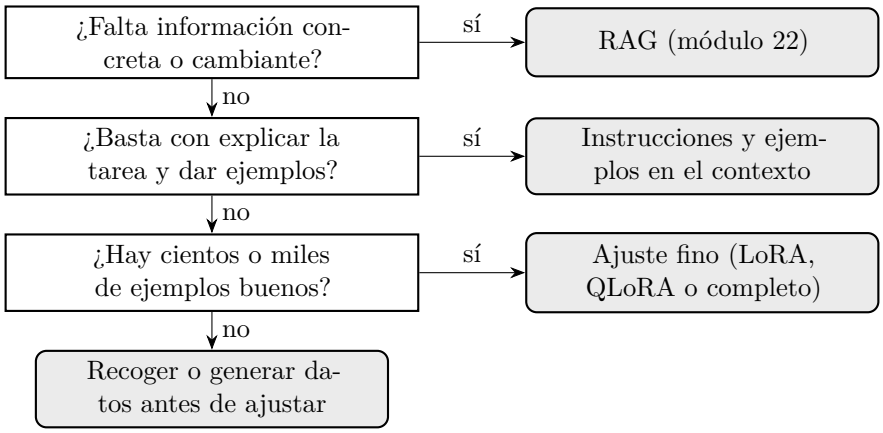


Figura 6.1: Diagrama de decisión orientativo para adaptar un modelo de lenguaje. Las preguntas (recuadros rectos) se recorren de arriba abajo; las respuestas están sombreadas.

- Dettmers *et al.* [11], §3 (NF4, doble cuantización y optimizadores paginados), §4 (comparación con el ajuste completo) y §8 (limitaciones).
- Módulo 19, cuantización por bloques, y módulo 17, contabilidad de memoria del entrenamiento.
- CS336 [7], tarea 1, §3.5 (contabilidad de parámetros y memoria de un Transformer).
- Jurafsky y Martin [19], cap. 8, y cap. 11 «Information Retrieval and RAG» como enlace con el módulo 22.

Ejercicios del tema 6

Ejercicio 6.1 ⋆ Una empresa quiere que su asistente responda conforme a la normativa interna, que cambia cada mes. ¿Ajuste fino o RAG? Justifíquelo con dos argumentos.

Ejercicio 6.2 ⋆⋆ Con el libro de códigos del ejemplo 6.4, $\mathcal{Q} = \{-1; -0{,}2770; 0{,}2770; 1\}$, cuantice el bloque $w = (0{,}9; -0{,}3; 0{,}1; -1{,}8)$ según la definición 6.1. Calcule los valores descuantizados, el error de cada peso y la norma euclídea del error. ¿Qué peso sale peor parado y qué propiedad de NF4 lo evitaría?

Ejercicio 6.3 ⋆⋆ Un modelo de $65 \cdot 10^9$ parámetros se cuantiza a 4 bits en bloques de 64. Calcule la memoria de la base (en GB, con $1\,\text{GB} = 10^9$ bytes) sin doble cuantización y con

ella, usando la proposición 6.5 con $B_2 = 256$, y el ahorro.

Ejercicio 6.4 ⋆⋆ Estime la memoria de pesos y optimizador para un modelo de $7 \cdot 10^9$ parámetros: (a) ajuste completo con Adam a 16 bytes por parámetro; (b) QLoRA con la base en 4 bits y $2 \cdot 10^7$ parámetros LoRA a 16 bytes por parámetro. ¿Cuántas veces menos memoria usa QLoRA?

Ejercicio 6.5 ⋆⋆ Un Transformer tiene vocabulario $V = 50\,257$, $d = 1600$, 48 capas y $d_{\text{ff}} = 4288$. Cada capa tiene 4 matrices de atención de $d \times d$, una red de alimentación SwiGLU con 3 matrices de $d \times d_{\text{ff}}$ y 2 normalizaciones RMSNorm de d pesos. Hay una RMSNorm final, los *embeddings* de entrada y salida no se comparten y no hay sesgos. (a) Cuente los parámetros N. (b) Con LoRA de rango $r = 8$ en las cuatro matrices de atención de todas las capas, ¿cuántos parámetros se entrenan y qué porcentaje de N representan? (c) Compare la memoria del ajuste completo (16 bytes por parámetro) con la de LoRA sobre una base en bf16 (2 bytes por parámetro) y adaptadores a 16 bytes por parámetro.

Fuente: adaptado de Stanford CS336, primavera de 2026, tarea 1 [7, §3.5], problema `transformer_accounting`, apartado (a); los apartados (b) y (c) son una ampliación.

Ejercicio 6.6 ⋆⋆⋆ Demuestre que LoRA de rango r entrena menos parámetros que el ajuste completo de una matriz $d \times k$ si y solo si $r < dk/(d+k)$. Calcule el rango límite para matrices de 4096×4096 y de 1024×4096. ¿Por qué en la práctica se usan rangos mucho menores?

Apéndice A

Soluciones de los ejercicios

Cada solución se presenta en cinco partes: datos, planteamiento, desarrollo, resultado y comprobación. En los pasos intermedios se redondea a cuatro cifras significativas salvo que se indique otra cosa; los resultados exactos se dan como fracción cuando existe. Las memorias se expresan con $1\,\mathrm{GB} = 10^9$ bytes.

A.1 Tema 1

Solución del ejercicio 1.1

Planteamiento. el cuerpo parte de θ_0, que ya es una buena solución; la tasa de aprendizaje controla cuánto se aleja de ella (observación 1.4).

Desarrollo.

1. Con tasas pequeñas, el cuerpo se adapta poco a poco y la cabeza, que parte de cero, aprende a usar las representaciones preentrenadas: la exactitud mejora respecto a la sonda lineal.
2. Al aumentar la tasa, los primeros pasos, con gradientes grandes procedentes de una cabeza aleatoria, desplazan mucho los parámetros del cuerpo y destruyen las representaciones aprendidas en la tarea de origen.
3. Con un conjunto de destino pequeño, el modelo pierde entonces la ventaja de la transferencia: se comporta como una red entrenada desde una inicialización arbitraria con pocos datos, y la exactitud baja. Con tasas aún mayores, el entrenamiento puede volverse inestable.

Resultado. **la exactitud sube al principio y después cae**, porque una tasa grande anula la regularización implícita que supone permanecer cerca de θ_0. Es una forma de olvido catastrófico (tema 3).

Comprobación. la receta de [28, §14.2] usa una tasa pequeña en el cuerpo y una diez veces mayor solo en la cabeza nueva, y ULMFiT reduce aún más la tasa en las capas inferiores.

Solución del ejercicio 1.2

Planteamiento. aplicar la proposición 1.6 y las definiciones del tema.

Desarrollo.

(a) **Falsa.** La convexidad garantiza que todo mínimo local es global, pero no la unicidad. Sumar el mismo vector w a todas las filas de V y la misma constante a todas las componentes de b suma a cada logit $w \cdot \phi + \mathrm{cte}$, igual para todas las clases, y no cambia la softmax (módulo 1, tema 4): si existe un minimizador, existen infinitos. Además, si los datos son separables, el mínimo no se alcanza.

(b) **Falsa.** Con una capa oculta, los logits dependen de los productos de los pesos de las dos capas y de una no linealidad; la pérdida no es convexa (por ejemplo, permutar las neuronas ocultas da otro mínimo con el mismo valor, y el punto medio entre ambos no lo es en general).

(c) **Falsa.** Por la definición 1.7, $\eta^{l-1} = \eta^l/2{,}6 < \eta^l$: la mayor es la de la última capa.

Resultado. **(a) falsa; (b) falsa; (c) falsa.**

Solución del ejercicio 1.3

Datos. $L = 12$, $\eta^{12} = 0{,}01$, factor 2,6.

Planteamiento. $\eta^l = \eta^L/2{,}6^{\,L-l}$.

Desarrollo.

1. $\eta^{11} = 0{,}01/2{,}6 = 3{,}846 \cdot 10^{-3}$.
2. $\eta^{10} = 0{,}01/2{,}6^2 = 0{,}01/6{,}76 = 1{,}479 \cdot 10^{-3}$.
3. $\eta^1 = 0{,}01/2{,}6^{11} = 0{,}01/36\,703 = 2{,}725 \cdot 10^{-7}$.
4. Se busca el menor L con $0{,}01/2{,}6^{L-1} < 10^{-6}$, es decir, $2{,}6^{L-1} > 10^4$, o $L - 1 > \ln 10^4/\ln 2{,}6 = 9{,}210/0{,}9555 = 9{,}639$.
5. El menor entero es $L - 1 = 10$, luego $L = 11$.

Resultado. $\eta^{11} = 3{,}846 \cdot 10^{-3}$**,** $\eta^{10} = 1{,}479 \cdot 10^{-3}$**,** $\eta^1 = 2{,}725 \cdot 10^{-7}$**; basta con** $L = 11$ **capas.**

Comprobación. con $L = 10$, $\eta^1 = 0{,}01/2{,}6^9 = 0{,}01/5430 = 1{,}842 \cdot 10^{-6} > 10^{-6}$; con $L = 11$, $0{,}01/14\,117 = 7{,}084 \cdot 10^{-7} < 10^{-6}$.

Solución del ejercicio 1.4

Datos. $T = 1000$, cf $= 0{,}1$, $\rho = 32$, $\eta_{\max} = 0{,}01$.

Planteamiento. $c = \lfloor 1000 \cdot 0{,}1 \rfloor = 100$; $c(1/\text{cf} - 1) = 100 \cdot 9 = 900$; $\eta_t = 0{,}01\,(1 + 31p_t)/32$.

Desarrollo.

1. $t = 0$: $p = 0$, $\eta = 0{,}01/32 = 3{,}125 \cdot 10^{-4}$.
2. $t = 50$: $p = 50/100 = 0{,}5$, $\eta = 0{,}01\,(1 + 15{,}5)/32 = 0{,}01 \cdot 16{,}5/32 = 5{,}156 \cdot 10^{-3}$.
3. $t = 100$: $p = 1 - 0/900 = 1$, $\eta = 0{,}01 \cdot 32/32 = 0{,}01$.
4. $t = 550$: $p = 1 - 450/900 = 0{,}5$, $\eta = 5{,}156 \cdot 10^{-3}$.
5. $t = 1000$: $p = 1 - 900/900 = 0$, $\eta = 3{,}125 \cdot 10^{-4}$.

Resultado. $3{,}125 \cdot 10^{-4}$**;** $5{,}156 \cdot 10^{-3}$**;** $0{,}01$**;** $5{,}156 \cdot 10^{-3}$**;** $3{,}125 \cdot 10^{-4}$**.**

Comprobación. la tasa sube en 100 iteraciones y baja en 900, con valores simétricos en p ($t = 50$ y $t = 550$ tienen $p = 0{,}5$), como predice la proposición 1.9.

Solución del ejercicio 1.5

Datos. $\eta(t) = \eta_{\max}(1 + p(t)(\rho - 1))/\rho$, con p lineal a trozos; $c = T\,\text{cf}$.

Planteamiento. por linealidad de la integral, basta calcular el valor medio de p.

Desarrollo.

1. En $[0, c]$, $p(t) = t/c$ y $\int_0^c p\,dt = c/2$.
2. En $[c, T]$, p decrece linealmente de 1 a 0 (proposición 1.9) y $\int_c^T p\,dt = (T-c)/2$, el área de un triángulo de base $T - c$ y altura 1.
3. Valor medio de p: $\frac{1}{T}\left(\frac{c}{2} + \frac{T-c}{2}\right) = \frac{1}{2}$, independiente de c y, por tanto, de cf.
4. Valor medio de η: $\frac{\eta_{\max}}{\rho}\left(1 + \frac{1}{2}(\rho - 1)\right) = \eta_{\max}\frac{\rho+1}{2\rho}$.
5. Con los valores por defecto: $0{,}01 \cdot 33/64 = 5{,}156 \cdot 10^{-3}$.

Resultado. $\bar{\eta} = \eta_{\max}(\rho + 1)/(2\rho)$**; con los valores por defecto,** $5{,}156 \cdot 10^{-3}$**.**

Comprobación. coincide con el valor en $p = 0{,}5$ del ejercicio 1.4, como corresponde a una función afín de p cuya media es 0,5.

A.2 Tema 2

Solución del ejercicio 2.1

Planteamiento. comparar lo que aprendió el modelo base (la distribución del corpus) con lo que se le pide (un turno de respuesta corto que termina).

Desarrollo.

1. El preentrenamiento minimiza la entropía cruzada sobre texto continuo: el modelo aprende $p(x_t \mid x_{1:t-1})$ tal como aparece en el corpus.
2. En ese corpus casi no aparece el patrón «p: … r:» seguido de una respuesta corta y un cierre. La continuación más probable es más texto con el aspecto del corpus, no «paris».
3. El dato «París es la capital de Francia» está en los pesos, pero asociado a frases del corpus, no a la pregunta en formato de chat.
4. El modelo base nunca recibió un token de fin de turno como objetivo: aunque empezara bien, no sabría parar.

Resultado. **le falta el formato, no el conocimiento.** El ajuste supervisado le enseña a responder en ese formato y a cerrar el turno con el token de fin.

Comprobación. es la hipótesis de la alineación superficial (LIMA). En la práctica, 600 pasos de SFT bastan para que el mismo mini-GPT responda y emita el token de fin en el 100 % de los ejemplos de prueba.

Solución del ejercicio 2.2

Planteamiento. recordar cómo calcula PyTorch la media de la entropía cruzada con `ignore_index`.

Desarrollo.

1. Con reducción media, PyTorch calcula la suma de las pérdidas de las posiciones válidas dividida por el número de posiciones válidas.
2. Una posición con etiqueta -100 no suma en el numerador ni cuenta en el denominador, y su gradiente respecto a los logits es nulo (proposición 2.7 con $m_t = 0$).

3. El relleno solo sirve para igualar longitudes dentro de un lote; esos tokens no forman parte de ningún ejemplo.
4. Sin -100, el modelo aprendería a predecir relleno y la media se diluiría con posiciones irrelevantes.

Resultado. **la posición se ignora por completo (numerador, denominador y gradiente)**; en el relleno se usa porque esos tokens no son datos.

Comprobación. si todas las etiquetas de la instrucción y del relleno valen -100, la pérdida coincide con la definición 2.4.

Solución del ejercicio 2.3

Datos. instrucción (2,1; 1,8; 0,9; 1,1); respuesta (0,6; 0,2; 0,4); $T = 7$, $\sum_t m_t = 3$.

Planteamiento. sin máscara se divide la suma de las 7 pérdidas entre 7; con máscara, la suma de las de la respuesta entre 3.

Desarrollo.

1. Suma de la instrucción: $2{,}1 + 1{,}8 + 0{,}9 + 1{,}1 = 5{,}9$.
2. Suma de la respuesta: $0{,}6 + 0{,}2 + 0{,}4 = 1{,}2$.
3. Sin máscara: $(5{,}9 + 1{,}2)/7 = 7{,}1/7 = 1{,}014$.
4. Con máscara: $1{,}2/3 = 0{,}400$.

Resultado. **sin máscara** 1,014 **nats; con máscara** 0,400 **nats.**

Comprobación. 1,014 está entre la media de la instrucción ($5{,}9/4 = 1{,}475$) y la de la respuesta (0,400), más cerca de la primera porque tiene más tokens.

Solución del ejercicio 2.4

Datos. ejemplo 1: $[5, 9, 2]$ y $[7, 3, 0]$; ejemplo 2: $[4, 8]$ y $[6, 0]$; relleno $= 1$.

Planteamiento. la etiqueta de la posición t es el token $t + 1$; la máscara vale 1 si esa etiqueta pertenece a la respuesta.

Desarrollo.

1. Concatenar: $[5, 9, 2, 7, 3, 0]$ (6 tokens) y $[4, 8, 6, 0]$ (4 tokens). Longitud máxima: 6.
2. Rellenar: $[4, 8, 6, 0, 1, 1]$.
3. `input_ids` (sin el último token): $[5, 9, 2, 7, 3]$ y $[4, 8, 6, 0, 1]$.
4. `labels` (sin el primer token): $[9, 2, 7, 3, 0]$ y $[8, 6, 0, 1, 1]$.
5. Máscara. Ejemplo 1: 9 y 2 son de la instrucción; 7, 3 y 0, de la respuesta: $[0, 0, 1, 1, 1]$. Ejemplo 2: 8 es de la instrucción; 6 y 0, de la respuesta; los dos 1 son relleno: $[0, 1, 1, 0, 0]$.

Resultado. **máscaras** $[0, 0, 1, 1, 1]$ **y** $[0, 1, 1, 0, 0]$**: puntúan 3 y 2 posiciones.**

Comprobación. cada ejemplo puntúa tantas posiciones como tokens tiene su respuesta, incluido el de fin.

Solución del ejercicio 2.5

Datos. tamaño 2, $c = 1$, 2 pasos de acumulación y

$$\ln p^{(1)} = (-2{,}0;\ -1{,}5;\ -0{,}5;\ -1{,}0;\ -0{,}1), \quad m^{(1)} = (0, 0, 1, 1, 1);$$
$$\ln p^{(2)} = (-1{,}2;\ -0{,}4;\ -0{,}2;\ -3{,}0;\ -3{,}0), \quad m^{(2)} = (0, 1, 1, 0, 0).$$

Planteamiento. sumar $-\ln p$ donde $m = 1$ y dividir por tamaño, c y pasos de acumulación (proposición 2.12).

Desarrollo.

1. Secuencia 1: $-(-0{,}5 - 1{,}0 - 0{,}1) = 1{,}6$.
2. Secuencia 2: $-(-0{,}4 - 0{,}2) = 0{,}6$.
3. Suma enmascarada: $1{,}6 + 0{,}6 = 2{,}2$.
4. Entre el tamaño del microlote y c: $2{,}2/2/1 = 1{,}1$.
5. Entre los pasos de acumulación: $1{,}1/2 = 0{,}55$.
6. $J_{\text{tok}} = 2{,}2/(3 + 2) = 0{,}44$.

Resultado. **pérdida del microlote** $0{,}55$**; media por token** $0{,}44$**.**

Comprobación. con dos microlotes iguales, $0{,}55 + 0{,}55 = 1{,}1$, la pérdida por secuencia sin acumulación. Las dos normalizaciones difieren porque las respuestas tienen longitudes distintas (3 y 2).

Solución del ejercicio 2.6

Datos. ejemplos de 7, 5 y 9 tokens; un fin tras cada uno; trozos de 5; alternativa con relleno a 10.

Planteamiento. contar los tokens totales, dividir entre la longitud del trozo y comparar posiciones útiles con posiciones totales.

Desarrollo.

1. Tokens con fin: $8 + 6 + 10 = 24$.
2. Trozos completos: $24 = 4 \cdot 5 + 4$. Salen 4 secuencias y se pierden 4 tokens.
3. Empaquetando se aprovechan $20/24 = 5/6 = 0{,}8333$ de los tokens, y todas las posiciones de cada secuencia son reales.
4. Con relleno: $3 \cdot 10 = 30$ posiciones, de las que 24 son reales: $24/30 = 0{,}80$.

Resultado. **4 secuencias y 4 tokens perdidos (**$5/6 \approx 83\,\%$ **aprovechado); con relleno, solo el** $80\,\%$ **de las posiciones son útiles.**

Comprobación. $4 \cdot 5 + 4 = 24$ y $24 + 6 = 30$.

Solución del ejercicio 2.7

Datos. secuencia $u^{(1)}, y^{(1)}, \ldots, u^{(K)}, y^{(K)}$; máscara 1 exactamente en los tokens de las $y^{(k)}$.

Planteamiento. aplicar la regla de la cadena a la secuencia completa y agrupar los factores por segmentos, como en la proposición 2.5.

Desarrollo.

1. El denominador es $\sum_t m_t = \sum_k m_k$.
2. El numerador es $\sum_{t:m_t=1} \ln p_\theta(x_t \mid x_{1:t-1})$. Las posiciones con $m_t = 1$ son las de las respuestas, y para el token j de $y^{(k)}$ el contexto $x_{1:t-1}$ es $(h^{(k)}, y^{(k)}_{1:j-1})$.
3. Agrupando por respuestas: $\sum_k \sum_{j=1}^{m_k} \ln p_\theta(y^{(k)}_j \mid h^{(k)}, y^{(k)}_{1:j-1})$.
4. Por la regla de la cadena condicionada a $h^{(k)}$, cada suma interior es $\ln p_\theta(y^{(k)} \mid h^{(k)})$.
5. Sustituyendo y cambiando el signo se obtiene la fórmula del enunciado.

Resultado. $\mathcal{L}_{\text{SFT}} = -\frac{1}{\sum_k m_k} \sum_k \ln p_\theta(y^{(k)} \mid h^{(k)})$**.** El ajuste maximiza la verosimilitud de cada respuesta del asistente dado todo el historial anterior, incluidas sus propias respuestas previas; los turnos del usuario solo actúan como contexto.

Comprobación. con $K = 1$ se recupera la proposición 2.5, con $h^{(1)} = u^{(1)}$.

A.3 Tema 3

Solución del ejercicio 3.1

Planteamiento. repasar los remedios de la tema 3 y de los temas 1, 4 y 5.

Desarrollo.

1. Sobre los datos: repetición de datos generales (definición 3.10).
2. Sobre la pérdida: penalizar el alejamiento de θ_0 (muelle, EWC) o de las salidas del modelo de referencia (divergencia KL, módulo 21).
3. Sobre el optimizador: tasa pequeña, pocas épocas, tasas discriminativas y descongelación gradual (tema 1).
4. Sobre los parámetros entrenados: métodos PEFT y LoRA, que dejan θ_0 congelado.

Resultado. **solo los métodos que congelan θ_0 (PEFT, LoRA) permiten recuperar exactamente el modelo base**: basta con desactivar el adaptador, porque los pesos originales no se han modificado.

Comprobación. en el ejemplo 3.2, desactivar el adaptador LoRA devuelve la pérdida de texto exacta del modelo base, 0,192 nats.

Solución del ejercicio 3.2

Datos. $\theta_0 = 1$; tarea $(\theta - 3)^2$; muelle $\lambda(\theta - 1)^2$; $\lambda = 2$.

Planteamiento. derivar e igualar a cero; la pérdida es una parábola abierta hacia arriba.

Desarrollo.

1. $\mathcal{L}'(\theta) = 2(\theta - 3) + 2\lambda(\theta - 1)$.
2. $\mathcal{L}' = 0 \iff \theta - 3 + \lambda\theta - \lambda = 0 \iff \theta(1 + \lambda) = 3 + \lambda \iff \theta^* = (3 + \lambda)/(1 + \lambda)$.
3. $\mathcal{L}'' = 2 + 2\lambda > 0$: es un mínimo.
4. $\lambda = 2$: $\theta^* = 5/3 \approx 1{,}667$.
5. Olvido: $(5/3 - 1)^2 = (2/3)^2 = 4/9 \approx 0{,}4444$.

6. Error en la tarea: $(5/3-3)^2=(-4/3)^2=16/9\approx 1{,}778$.
7. Límite: $\theta^*=(3/\lambda+1)/(1/\lambda+1)\to 1$ cuando $\lambda\to\infty$.

Resultado. $\theta^*=(3+\lambda)/(1+\lambda)$**; con** $\lambda=2$**,** $\theta^*=5/3$**, olvido** $4/9$ **y error** $16/9$**. Si** $\lambda\to\infty$**,** $\theta^*\to 1$**: no olvida, pero tampoco aprende.**

Comprobación. por el corolario 3.4, $\sqrt{4/9}+\sqrt{16/9}=2/3+4/3=2=|3-1|$.

Solución del ejercicio 3.3

Datos. $\theta_A=(1,1)$; $F=(4;\,0{,}25)$; $\lambda=2$; $\mathcal{L}_B=(\theta_1-3)^2+(\theta_2-3)^2$.

Planteamiento. la pérdida se separa por coordenadas: $(\theta_i-3)^2+\frac{\lambda F_i}{2}(\theta_i-1)^2$. Por la proposición 3.3 con muelle $\frac{\lambda F_i}{2}$, $\theta_i^*=(3+\frac{\lambda F_i}{2})/(1+\frac{\lambda F_i}{2})$.

Desarrollo.

1. Coordenada 1: $\lambda F_1/2=2\cdot 4/2=4$; $\theta_1^*=(3+4)/(1+4)=7/5=1{,}4$.
2. Coordenada 2: $\lambda F_2/2=2\cdot 0{,}25/2=0{,}25$; $\theta_2^*=3{,}25/1{,}25=2{,}6$.
3. Penalización: $\frac{1}{2}\big(4\cdot 0{,}4^2+0{,}25\cdot 1{,}6^2\big)=\frac{1}{2}(0{,}64+0{,}64)=0{,}64$.
4. Error en la tarea nueva: $(1{,}4-3)^2+(2{,}6-3)^2=2{,}56+0{,}16=2{,}72$.

Resultado. $\theta^*=(1{,}4;\,2{,}6)$**; penalización** $0{,}64$**;** $\mathcal{L}_B(\theta^*)=2{,}72$**.** Se mueve mucho más θ_2, cuya información de Fisher es 16 veces menor: la tarea antigua apenas depende de él.

Comprobación. derivada en la coordenada 1: $2(1{,}4-3)+2\cdot 4\cdot(1{,}4-1)=-3{,}2+3{,}2=0$; en la 2: $2(2{,}6-3)+2\cdot 0{,}25\cdot(2{,}6-1)=-0{,}8+0{,}8=0$.

Solución del ejercicio 3.4

Datos. $\mathcal{L}_B=(\theta-3)^2$, $\mathcal{L}_A=(\theta-1)^2$, $\beta=0{,}25$.

Planteamiento. derivar la mezcla e igualar a cero.

Desarrollo.

1. $\frac{d}{d\theta}\big[(1-\beta)(\theta-3)^2+\beta(\theta-1)^2\big]=2(1-\beta)(\theta-3)+2\beta(\theta-1)$.
2. Igualando a cero: $\theta=(1-\beta)\cdot 3+\beta\cdot 1=3-2\beta$.
3. $\beta=0{,}25$: $\theta^*=3-0{,}5=2{,}5$.
4. Con $\lambda=\beta/(1-\beta)$, la proposición 3.3 da $\frac{3+\lambda}{1+\lambda}=\frac{3(1-\beta)+\beta}{(1-\beta)+\beta}=3-2\beta$, tras multiplicar numerador y denominador por $1-\beta$.
5. Para $\beta=0{,}25$, $\lambda=1/3$ y $(3+1/3)/(1+1/3)=(10/3)/(4/3)=2{,}5$.

Resultado. $\theta^*(\beta)=3-2\beta$**; con** $\beta=0{,}25$**,** $\theta^*=2{,}5$**, igual que el muelle con** $\lambda=1/3$**.**

Comprobación. la mezcla es $(1-\beta)\big[\mathcal{L}_B+\frac{\beta}{1-\beta}\mathcal{L}_A\big]$, y el factor positivo $1-\beta$ no cambia el minimizador.

Solución del ejercicio 3.5

Datos. $p_z(1)=\sigma(z)$, $p_z(0)=1-\sigma(z)$; $\sigma'(z)=\sigma(z)(1-\sigma(z))$.

Planteamiento. calcular la puntuación y su segundo momento (definición 3.7).

Desarrollo.

1. $\ln p_z(x) = x \ln \sigma(z) + (1-x)\ln(1-\sigma(z))$ para $x \in \{0,1\}$.
2. Puntuación: $\frac{d}{dz}\ln p_z(x) = x\frac{\sigma'}{\sigma} - (1-x)\frac{\sigma'}{1-\sigma} = x(1-\sigma) - (1-x)\sigma = x - \sigma(z)$.
3. $\mathcal{I}(z) = \mathbb{E}[(x-\sigma)^2] = \mathrm{Var}(x) = \sigma(z)(1-\sigma(z))$, porque $\mathbb{E}[x] = \sigma(z)$.
4. $z = 0$: $\sigma = 0{,}5$ y $\mathcal{I} = 0{,}25$.
5. $z = \ln 9$: $\sigma = 1/(1+1/9) = 0{,}9$ y $\mathcal{I} = 0{,}9 \cdot 0{,}1 = 0{,}09$.

Resultado. $\mathcal{I}(z) = \sigma(z)(1-\sigma(z))$**; vale** $0{,}25$ **en** $z = 0$ **y** $0{,}09$ **en** $z = \ln 9$**.** El parámetro con $z = 0$ quedaría unas 2,8 veces más rígido: una predicción incierta es más sensible a cambios del logit.

Comprobación. por la proposición 3.8, $-\frac{d^2}{dz^2}\ln p_z(x) = \frac{d}{dz}\sigma(z) = \sigma(1-\sigma)$, que no depende de x y coincide con el resultado.

Solución del ejercicio 3.6

Datos. $\mathcal{L}_B = \frac{1}{2}(\theta-\theta_B)^\top H(\theta-\theta_B)$, $H = Q\Lambda Q^\top$; penalización $\frac{\lambda}{2}\|\theta-\theta_0\|^2$.

Planteamiento. anular el gradiente y diagonalizar con la base de autovectores.

Desarrollo.

1. (a) El gradiente es $H(\theta-\theta_B) + \lambda(\theta-\theta_0)$. Se anula si $(H+\lambda I)\theta = H\theta_B + \lambda\theta_0$.
2. $H + \lambda I$ tiene autovalores $h_i + \lambda > 0$, luego es definida positiva e invertible, y la función es estrictamente convexa: $\theta^* = (H+\lambda I)^{-1}(H\theta_B + \lambda\theta_0)$ es el único minimizador.
3. (b) Restando $\theta_0 = (H+\lambda I)^{-1}(H+\lambda I)\theta_0$: $\theta^* - \theta_0 = (H+\lambda I)^{-1}H(\theta_B - \theta_0)$.
4. Con $H = Q\Lambda Q^\top$, $(H+\lambda I)^{-1}H = Q(\Lambda+\lambda I)^{-1}\Lambda Q^\top$. Multiplicando por $q_i^\top$: $q_i^\top(\theta^*-\theta_0) = \frac{h_i}{h_i+\lambda}\, q_i^\top(\theta_B - \theta_0)$.
5. (c) H ya es diagonal, $\theta_0 = 0$: $\theta_1^* = \frac{10}{11}\cdot 1 = 0{,}9091$ y $\theta_2^* = \frac{0{,}1}{1{,}1}\cdot 1 = 0{,}09091$.

Resultado. $\theta^* = (H+\lambda I)^{-1}(H\theta_B + \lambda\theta_0)$**; factor** $h_i/(h_i+\lambda)$ **por dirección; en (c),** $\theta^* = (0{,}9091;\ 0{,}09091)$**.** En la dirección de curvatura alta, importante para la tarea nueva, el modelo casi alcanza θ_B; en la de curvatura baja, la penalización domina y el parámetro apenas se aleja de θ_0.

Comprobación. con $\lambda = 0$ el factor es 1 ($\theta^* = \theta_B$); con $\lambda \to \infty$ tiende a 0 ($\theta^* \to \theta_0$), como en el corolario 3.4.

A.4 Tema 4

Solución del ejercicio 4.1

Datos. $h = 768$; $K = 2$ clases con sesgo; cuerpo de $1{,}1 \cdot 10^8$ parámetros.

Planteamiento. la cabeza tiene $Kh + K$ parámetros (definición 1.5).

Desarrollo.

1. (a) $2 \cdot 768 + 2 = 1538$ parámetros; fracción $1538/(1{,}1\cdot 10^8 + 1538) = 1{,}398 \cdot 10^{-5}$, un $0{,}0014\,\%$.

2. (b) Es un método selectivo: se entrena un subconjunto de los parámetros (solo la cabeza). Es la extracción de características o sonda lineal.
3. (c) En general, menos exactitud: las representaciones no se adaptan a la tarea. La diferencia es pequeña si la tarea de destino se parece a la de origen, de modo que las representaciones ya separan bien las clases, o si hay muy pocos datos, porque entonces el ajuste completo sobreajusta.

Resultado. **1538 parámetros (0,0014 %); método selectivo; exactitud en general menor, con diferencia pequeña si las tareas se parecen o hay pocos datos.**

Comprobación. por la proposición 1.6, este problema es convexo, lo que lo hace estable y barato aunque menos expresivo.

Solución del ejercicio 4.2

Datos. $L = 12$; $d = 768$; $m = 64$; 2 adaptadores por capa; modelo de $1{,}1 \cdot 10^8$ parámetros; 20 tareas.

Planteamiento. por la proposición 4.4, cada adaptador tiene $2md + m + d$ parámetros.

Desarrollo.

1. Por adaptador: $2 \cdot 64 \cdot 768 + 64 + 768 = 98\,304 + 832 = 99\,136$.
2. Total: $2 \cdot 12 \cdot 99\,136 = 2\,379\,264$.
3. Porcentaje: $2\,379\,264 / 1{,}1 \cdot 10^8 = 0{,}02163$, un 2,16 %.
4. (b) Con adaptadores: un modelo base y 20 juegos de adaptadores, $1{,}1 \cdot 10^8 + 20 \cdot 2\,379\,264 = 1{,}1 \cdot 10^8 + 47\,585\,280 = 1{,}576 \cdot 10^8$ parámetros.
5. Con copias completas: $20 \cdot 1{,}1 \cdot 10^8 = 2{,}2 \cdot 10^9$ parámetros.

Resultado. $2\,379\,264$ **parámetros (2,16 %); para 20 tareas,** $1{,}58 \cdot 10^8$ **frente a** $2{,}2 \cdot 10^9$ **parámetros, unas 14 veces menos.**

Comprobación. $2{,}2 \cdot 10^9 / 1{,}576 \cdot 10^8 = 13{,}96$. El porcentaje es del orden del 3,6 % que citan Houlsby *et al.*, que usan otros tamaños de cuello.

Solución del ejercicio 4.3

Datos. $L = 24$; $d = 1024$; $p = 20$.

Planteamiento. definición 4.6 (pd) y definición 4.7 ($2Lpd$).

Desarrollo.

1. Ajuste de *prompt*: $20 \cdot 1024 = 20\,480$.
2. Ajuste de prefijo: $2 \cdot 24 \cdot 20 \cdot 1024 = 983\,040$.
3. Cociente: $983\,040 / 20\,480 = 48$.

Resultado. $20\,480$ **frente a** $983\,040$ **parámetros; el prefijo tiene** $48 = 2L$ **veces más.**

Comprobación. el cociente es $2Lpd/(pd) = 2L$: un par de claves y valores por capa frente a un único *embedding* de entrada.

Solución del ejercicio 4.4

Datos. $\theta_0 = (1, 0, 2)$; $P = (1, 2, -1)^\top$; $\theta^\star = (2, 3, 1)$; $\mathcal{L} = \|\theta - \theta^\star\|^2$.

Planteamiento. $\mathcal{L}(z) = \|\theta_0 - \theta^\star + zP\|^2$ es una parábola en z.

Desarrollo.

1. $\theta_0 - \theta^\star = (-1, -3, 1)$; $\|P\|^2 = 1 + 4 + 1 = 6$; $P^\top(\theta_0 - \theta^\star) = -1 - 6 - 1 = -8$; $\|\theta_0 - \theta^\star\|^2 = 1 + 9 + 1 = 11$.
2. $\mathcal{L}(z) = 6z^2 - 16z + 11$. Mínimo en $z^* = 16/12 = 4/3$.
3. $\theta = \theta_0 + \frac{4}{3}P = (1 + \frac{4}{3},\ \frac{8}{3},\ 2 - \frac{4}{3}) = (7/3,\ 8/3,\ 2/3)$.
4. (b) $\mathcal{L}(4/3) = 6 \cdot \frac{16}{9} - \frac{64}{3} + 11 = \frac{32}{3} - \frac{64}{3} + \frac{33}{3} = \frac{1}{3}$. El ajuste completo alcanza $\theta = \theta^\star$ y pérdida 0.
5. (c) En $z = 0$, $\nabla_\theta \mathcal{L} = 2(\theta_0 - \theta^\star) = (-2, -6, 2)$ y $P^\top \nabla_\theta \mathcal{L} = -2 - 12 - 2 = -16$, que coincide con $\mathcal{L}'(0) = -16$.

Resultado. $z^* = 4/3$**,** $\theta = (7/3;\ 8/3;\ 2/3)$**; pérdida residual** $1/3$ **frente a 0; el gradiente vale** -16 **por los dos caminos.**

Comprobación. el residuo $\theta^\star - \theta = (-1/3, 1/3, 1/3)$ es ortogonal a P: $-\frac{1}{3} + \frac{2}{3} - \frac{1}{3} = 0$, como debe ocurrir en una proyección ortogonal.

Solución del ejercicio 4.5

Datos. $\theta = \theta_0 + Pz$, $P \in \mathbb{R}^{D \times d'}$.

Planteamiento. usar la definición de espacio columna y la proposición 4.11.

Desarrollo.

1. (a) $\{Pz : z \in \mathbb{R}^{d'}\}$ es, por definición, $\mathrm{Col}(P)$, un subespacio vectorial de dimensión rango P. Trasladado por θ_0 es el subespacio afín $\theta_0 + \mathrm{Col}(P)$.
2. (b) Un paso en z es $z \leftarrow z - \eta \nabla_z \mathcal{L} = z - \eta P^\top \nabla_\theta \mathcal{L}$. Multiplicando por P, θ cambia en $P \cdot (-\eta P^\top \nabla_\theta \mathcal{L}) = -\eta P P^\top \nabla_\theta \mathcal{L}$.
3. (c) Si $P^\top P = I$, la matriz $\Pi = PP^\top$ es simétrica e idempotente ($\Pi^2 = P(P^\top P)P^\top = \Pi$), con imagen $\mathrm{Col}(P)$: es el proyector ortogonal sobre $\mathrm{Col}(P)$.
4. El paso del ajuste completo es $-\eta \nabla_\theta \mathcal{L}$; su proyección es $\Pi(-\eta \nabla_\theta \mathcal{L}) = -\eta P P^\top \nabla_\theta \mathcal{L}$, que coincide con (b).

Resultado. **los modelos alcanzables forman** $\theta_0 + \mathrm{Col}(P)$**; el paso en el subespacio es** $-\eta P P^\top \nabla_\theta \mathcal{L}$**, la proyección ortogonal del paso completo cuando** P **tiene columnas ortonormales.**

Comprobación. en el ejercicio 4.4, $P/\sqrt{6}$ es unitario y $\Pi \nabla_\theta \mathcal{L} = \frac{-16}{6} P$, proporcional a P, como corresponde a una proyección sobre una recta.

A.5 Tema 5

Solución del ejercicio 5.1

Planteamiento. aplicar la definición 5.1, el corolario 5.9 y la proposición 5.4.

Desarrollo.

(a) **Falsa.** W no recibe actualizaciones: el optimizador solo contiene A y B. W' cambia porque cambia BA. Por eso desactivar el adaptador devuelve exactamente el modelo base (ejemplo 3.2).

(b) **Falsa.** Con $A = B = 0$, $\partial\mathcal{L}/\partial B = s\,G(Ax)^\top = 0$ y $\partial\mathcal{L}/\partial A = s\,B^\top G x^\top = 0$: ningún parámetro se mueve (ejercicio 5.7).

(c) **Verdadera.** W' tiene la misma forma que W y $W'x$ cuesta las mismas operaciones (proposición 5.4).

Resultado. **(a) falsa; (b) falsa; (c) verdadera.**

Comprobación. (c) solo vale tras fusionar; sin fusionar, cada capa calcula además $B(Ax)$, con $2r(d+k)$ operaciones más.

Solución del ejercicio 5.2

Datos. (a) $d = 1024$, $k = 4096$, $r = 8$. (b) 12 capas, 4 matrices de 768×768 por capa, $r = 8$.

Planteamiento. proposición 5.2: dk frente a $r(d+k)$.

Desarrollo.

1. (a) Completo: $1024 \cdot 4096 = 4\,194\,304$.
2. LoRA: $8 \cdot (1024 + 4096) = 8 \cdot 5120 = 40\,960$.
3. Porcentaje: $40\,960/4\,194\,304 = 9{,}766 \cdot 10^{-3}$, un 0,977 %.
4. (b) LoRA por matriz: $8 \cdot 1536 = 12\,288$. Total: $12 \cdot 4 \cdot 12\,288 = 589\,824$.
5. Matrices completas: $48 \cdot 768^2 = 48 \cdot 589\,824 = 28\,311\,552$.
6. Porcentaje: $589\,824/28\,311\,552 = 0{,}02083$, un 2,08 %.

Resultado. **(a)** 4 194 304 **parámetros; LoRA entrena** 40 960 **(**0,98 %**). (b)** 589 824 **frente a** 28 311 552**:** 2,08 %**.**

Comprobación. (a) $r(1/d + 1/k) = 8(1/1024 + 1/4096) = 8 \cdot 1{,}221 \cdot 10^{-3} = 9{,}766 \cdot 10^{-3}$. (b) con $d = k$ la fracción es $2r/d = 16/768 = 0{,}02083$; todo LoRA ocupa lo mismo que una sola matriz de 768×768.

Solución del ejercicio 5.3

Datos. $W = I_2$, $B = (1, 0)^\top$, $A = (2, 1)$, $r = 1$, $\alpha = 2$, $x = (1, 1)$.

Planteamiento. $W' = W + s\,BA$ con $s = \alpha/r$, y comprobación con $h = Wx + s\,B(Ax)$.

Desarrollo.

1. $s = 2/1 = 2$.
2. $BA = \begin{pmatrix} 1\cdot 2 & 1\cdot 1 \\ 0\cdot 2 & 0\cdot 1 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 0 & 0 \end{pmatrix}$; $2BA = \begin{pmatrix} 4 & 2 \\ 0 & 0 \end{pmatrix}$.
3. $W' = I + 2BA = \begin{pmatrix} 5 & 2 \\ 0 & 1 \end{pmatrix}$.
4. $W'x = (5 + 2;\ 0 + 1) = (7, 1)$.

Resultado. $W' = \begin{pmatrix} 5 & 2 \\ 0 & 1 \end{pmatrix}$ **y** $W'x = (7, 1)$**.**

Comprobación. sin fusionar: $Ax = 2 + 1 = 3$; $B(Ax) = (3, 0)$; $h = (1, 1) + 2 \cdot (3, 0) = (7, 1)$.

Solución del ejercicio 5.4

Datos. $s = 2$, $B = (0, 0)^\top$, $A = (1, 1)$, $G = (1, -1)$, $x = (2, 1)$.

Planteamiento. proposición 5.8.

Desarrollo.

1. $Ax = 1 \cdot 2 + 1 \cdot 1 = 3$ (escalar, porque $r = 1$).
2. $\partial \mathcal{L}/\partial B = 2 \cdot (1, -1)^\top \cdot 3 = (6, -6)^\top$.
3. $B^\top G = 0 \cdot 1 + 0 \cdot (-1) = 0$, luego $\partial \mathcal{L}/\partial A = 2 \cdot 0 \cdot x^\top = (0, 0)$.

Resultado. $\partial \mathcal{L}/\partial B = (6, -6)^\top$ **y** $\partial \mathcal{L}/\partial A = (0, 0)$: en el primer paso solo se mueve B.

Comprobación. dimensiones: $\partial \mathcal{L}/\partial B$ es $d \times r = 2 \times 1$ y $\partial \mathcal{L}/\partial A$, $r \times k = 1 \times 2$.

Solución del ejercicio 5.5

Datos. $\Delta W = \begin{pmatrix} 2 & 1 & 1 \\ 1 & 2 & -1 \end{pmatrix}$.

Planteamiento. los autovalores de $\Delta W \Delta W^\top$ son σ_i^2, sus autovectores los u_i, y $v_i = \Delta W^\top u_i / \sigma_i$; el teorema 5.11 da $\sigma_1 u_1 v_1^\top$.

Desarrollo.

1. $\Delta W \Delta W^\top = \begin{pmatrix} 4+1+1 & 2+2-1 \\ 2+2-1 & 1+4+1 \end{pmatrix} = \begin{pmatrix} 6 & 3 \\ 3 & 6 \end{pmatrix}$.
2. $(6 - \lambda)^2 - 9 = 0 \Rightarrow \lambda_1 = 9$, $\lambda_2 = 3$; $\sigma_1 = 3$, $\sigma_2 = \sqrt{3} = 1{,}732$.
3. Para $\lambda_1 = 9$: $-3u_1 + 3u_2 = 0 \Rightarrow u_1 = (1, 1)/\sqrt{2}$.
4. $v_1 = \Delta W^\top u_1 / 3 = (3, 3, 0)/(3\sqrt{2}) = (1, 1, 0)/\sqrt{2}$.
5. $\Delta W_1 = 3 \cdot \frac{1}{2} \begin{pmatrix} 1 & 1 & 0 \\ 1 & 1 & 0 \end{pmatrix} = \begin{pmatrix} 1{,}5 & 1{,}5 & 0 \\ 1{,}5 & 1{,}5 & 0 \end{pmatrix}$.
6. Error: $\|\Delta W - \Delta W_1\|_F = \sigma_2 = \sqrt{3} \approx 1{,}732$. Energía conservada: $9/(9 + 3) = 0{,}75$.
7. Como producto: $\sigma_1 u_1 v_1^\top = 1{,}5\,(1, 1)^\top (1, 1, 0)$; se toma $B = (1{,}5;\ 1{,}5)^\top$ y $A = (1, 1, 0)$.

Resultado. $\sigma = (3;\ \sqrt{3})$**;** $\Delta W_1 = \begin{pmatrix} 1{,}5 & 1{,}5 & 0 \\ 1{,}5 & 1{,}5 & 0 \end{pmatrix}$**; error** $\sqrt{3}$**; energía** 75 %**;** $B = (1{,}5;\ 1{,}5)^\top$**,** $A = (1, 1, 0)$**.**

Comprobación. $\Delta W - \Delta W_1 = \begin{pmatrix} 0{,}5 & -0{,}5 & 1 \\ -0{,}5 & 0{,}5 & -1 \end{pmatrix}$, de norma al cuadrado $0{,}25 \cdot 4 + 1 + 1 = 3$; y $\|\Delta W\|_F^2 = 12 = \sigma_1^2 + \sigma_2^2$.

Solución del ejercicio 5.6

Planteamiento. probar que la imagen de BA está contenida en la de B y buscar una matriz de rango 2.

Desarrollo.

1. Para todo $x \in \mathbb{R}^k$, $(BA)x = B(Ax)$ por asociatividad. Con $z = Ax \in \mathbb{R}^r$, $Bz = z_1 b_1 + \cdots + z_r b_r$, donde b_j son las columnas de B.
2. Luego $\operatorname{im}(BA) \subseteq \operatorname{Col}(B)$, generado por r vectores: $\operatorname{rango}(BA) \leq \operatorname{rango} B \leq r$.
3. Si $r < \min(d, k)$, existen matrices $d \times k$ de rango $\min(d, k) > r$ (la que tiene unos en la diagonal principal y ceros en el resto), que ningún BA puede igualar.
4. Ejemplo: $d = k = 2$, $r = 1$, $\Delta W = I_2$. Con $B = (b_1, b_2)^\top$ y $A = (a_1, a_2)$, $BA = \begin{pmatrix} b_1 a_1 & b_1 a_2 \\ b_2 a_1 & b_2 a_2 \end{pmatrix}$ y $\det(BA) = b_1 a_1 b_2 a_2 - b_1 a_2 b_2 a_1 = 0$, mientras que $\det I_2 = 1$.

Resultado. $\operatorname{rango}(BA) \leq r$**;** I_2 **no es representable con** $r = 1$**.**

Comprobación. el determinante nulo es una prueba independiente de que toda BA de 2×2 con $r = 1$ tiene rango a lo sumo 1.

Solución del ejercicio 5.7

Planteamiento. sustituir los valores iniciales en la proposición 5.8 y razonar por inducción sobre los pasos.

Desarrollo.

1. Con $A = 0$: $Ax = 0$ y $\partial \mathcal{L}/\partial B = s\, G \cdot 0^\top = 0$. Con $B = 0$: $B^\top G = 0$ y $\partial \mathcal{L}/\partial A = 0$. Vale para todo x y todo G.
2. Actualización: $A \leftarrow 0 - \eta \cdot 0 = 0$ y $B \leftarrow 0$. Por inducción sobre el número de pasos, $A = B = 0$ siempre.
3. Con Adam: si todos los gradientes son nulos, los momentos lo son y la actualización es nula (con $\varepsilon > 0$ en el denominador no hay división por cero).
4. Con $B = 0$ y $A \neq 0$: $\partial \mathcal{L}/\partial B = s\, G (Ax)^\top$, producto exterior de $G \in \mathbb{R}^d$ por $Ax \in \mathbb{R}^r$ multiplicado por $s \neq 0$.
5. Si $G \neq 0$ y $Ax \neq 0$, existen i con $G_i \neq 0$ y j con $(Ax)_j \neq 0$, y la entrada (i, j) vale $s\, G_i (Ax)_j \neq 0$.
6. Así B cambia en el primer paso y, desde el segundo, $B \neq 0$ y A también recibe gradiente.

Resultado. $(0, 0)$ **es un punto fijo del descenso de gradiente; con** $B = 0$ **y** $A \neq 0$**,** B **se mueve desde el primer paso.**

Comprobación. el ejercicio 5.4 es un caso numérico de la segunda parte: $\partial \mathcal{L}/\partial B = (6, -6)^\top \neq 0$ y $\partial \mathcal{L}/\partial A = 0$.

A.6 Tema 6

Solución del ejercicio 6.1

Planteamiento. la regla del tema 6: el ajuste cambia cómo responde el modelo; el contexto, qué sabe.

Desarrollo.

1. Coste de actualización: la normativa cambia cada mes. Con RAG basta con indexar los documentos nuevos; con ajuste fino habría que reentrenar cada mes, con riesgo de olvido (tema 3).
2. Fiabilidad: RAG pone el texto exacto de la norma en el contexto y permite citar la fuente; un modelo ajustado mezcla los datos en sus pesos y puede inventar detalles plausibles.

Resultado. **RAG**, por coste de actualización y por trazabilidad de las fuentes.

Comprobación. cuadro 6.2, fila «Conocimiento nuevo o cambiante».

Solución del ejercicio 6.2

Datos. $\mathcal{Q} = \{-1;\ -0{,}2770;\ 0{,}2770;\ 1\}$; $w = (0{,}9;\ -0{,}3;\ 0{,}1;\ -1{,}8)$.

Planteamiento. $c = \max|w_j|$, normalizar, redondear al nivel más próximo y descuantizar (definición 6.1). Los puntos medios entre niveles son $\pm 0{,}6385$ y 0.

Desarrollo.

1. $c = 1{,}8$. Normalizados: $w/c = (0{,}5;\ -0{,}1667;\ 0{,}05556;\ -1)$.
2. Niveles más próximos: $0{,}5 \to 0{,}2770$ (pues $0{,}5 < 0{,}6385$); $-0{,}1667 \to -0{,}2770$; $0{,}05556 \to 0{,}2770$; $-1 \to -1$.
3. Descuantizados: $1{,}8 \cdot 0{,}2770 = 0{,}4986$, luego $\hat{w} = (0{,}4986;\ -0{,}4986;\ 0{,}4986;\ -1{,}8)$.
4. Errores $w - \hat{w}$: $(0{,}4014;\ 0{,}1986;\ -0{,}3986;\ 0)$.
5. Norma: $\sqrt{0{,}1611 + 0{,}03944 + 0{,}1589 + 0} = \sqrt{0{,}3595} = 0{,}5995$.

Resultado. $\hat{w} = (0{,}4986;\ -0{,}4986;\ 0{,}4986;\ -1{,}8)$**; norma del error** 0,5995**.** El peor parado en términos relativos es $w_3 = 0{,}1$, que se convierte en 0,4986, casi cinco veces mayor: el libro de códigos no contiene el cero. NF4 incluye un nivel exactamente nulo (tema 6), con el que w_3 tendría error 0,1.

Comprobación. el máximo del bloque se representa sin error ($-1{,}8 \to -1{,}8$), como ocurre siempre con la escala absmax.

Solución del ejercicio 6.3

Datos. $N = 65 \cdot 10^9$; $b = 4$; $B_1 = 64$; $B_2 = 256$.

Planteamiento. proposición 6.2 y proposición 6.5; bytes = bits/8.

Desarrollo.

1. Sin doble cuantización: $4 + 32/64 = 4{,}5$ bits por parámetro; $65 \cdot 10^9 \cdot 4{,}5/8 = 36{,}56 \cdot 10^9$ bytes = 36,56 GB.
2. Con doble cuantización: $4 + 8/64 + 32/(64 \cdot 256) = 4 + 0{,}125 + 0{,}001953 = 4{,}127$ bits; $65 \cdot 10^9 \cdot 4{,}127/8 = 33{,}53$ GB.
3. Ahorro: 0,3730 bits por parámetro; $65 \cdot 10^9 \cdot 0{,}3730/8 = 3{,}031$ GB.

Resultado. 36,56 **GB sin doble cuantización,** 33,53 **GB con ella: unos 3 GB de ahorro.**

Comprobación. $36{,}56 - 33{,}53 = 3{,}03$. En una GPU de 48 GB, esos 3 GB son una parte apreciable del margen para activaciones y adaptadores.

Solución del ejercicio 6.4

Datos. $N = 7 \cdot 10^9$; ajuste completo a 16 B por parámetro; QLoRA con base a 0,5 B por parámetro y $N_{\text{LoRA}} = 2 \cdot 10^7$ a 16 B.

Planteamiento. proposición 6.7 con $b_{\text{ef}} = 4$.

Desarrollo.

1. (a) $7 \cdot 10^9 \cdot 16 = 112 \cdot 10^9$ B = 112 GB.
2. (b) Base: $7 \cdot 10^9 \cdot 0{,}5 = 3{,}5$ GB. Adaptadores: $2 \cdot 10^7 \cdot 16 = 3{,}2 \cdot 10^8$ B = 0,32 GB. Total: 3,82 GB.
3. Razón: $112/3{,}82 = 29{,}32$.

Resultado. **(a) 112 GB; (b)** 3,82 **GB; QLoRA usa unas** 29,3 **veces menos memoria.**

Comprobación. los pesos solos pasan de 14 GB (bf16) a 3,5 GB (un cuarto); el resto del ahorro procede de no guardar gradientes ni estado de Adam para W.

Solución del ejercicio 6.5

Datos. $V = 50\,257$; $d = 1600$; 48 capas; $d_{\text{ff}} = 4288$; $r = 8$.

Planteamiento. sumar por bloques; LoRA entrena $r(d+k)$ por matriz; memoria = parámetros $\times$ bytes.

Desarrollo.

1. Atención por capa: $4d^2 = 4 \cdot 2\,560\,000 = 10\,240\,000$.
2. Red de alimentación por capa: $3 \cdot 1600 \cdot 4288 = 20\,582\,400$.
3. RMSNorm por capa: $2 \cdot 1600 = 3200$.
4. Por capa: 30 825 600; 48 capas: 1 479 628 800.
5. *Embeddings*: $2 \cdot 50\,257 \cdot 1600 = 160\,822\,400$. Norma final: 1600.
6. $N = 1\,479\,628\,800 + 160\,822\,400 + 1600 = 1\,640\,452\,800 \approx 1{,}64 \cdot 10^9$.
7. (b) LoRA por matriz: $8 \cdot 3200 = 25\,600$. Total: $48 \cdot 4 \cdot 25\,600 = 4\,915\,200$, el $4\,915\,200/1\,640\,452\,800 = 0{,}2996\,\%$ de N.
8. (c) Ajuste completo: $1\,640\,452\,800 \cdot 16 = 26{,}25$ GB.
9. LoRA: base $1\,640\,452\,800 \cdot 2 = 3{,}281$ GB; adaptadores $4\,915\,200 \cdot 16 = 0{,}07864$ GB; total 3,360 GB.
10. Razón: $26{,}25/3{,}360 = 7{,}813$.

Resultado. **(a)** $N = 1\,640\,452\,800$**; (b)** 4 915 200 **parámetros LoRA** (0,30 %)**; (c)** 26,25 **GB frente a** 3,36 **GB, unas** 7,8 **veces menos.**

Comprobación. LoRA equivale al $2r/d = 16/1600 = 1\,\%$ de las matrices de atención: $48 \cdot 10\,240\,000 \cdot 0{,}01 = 4\,915\,200$.

Solución del ejercicio 6.6

Datos. matrices de 4096×4096 y 1024×4096.

Planteamiento. comparar $r(d+k)$ con dk.

Desarrollo.

1. Como $d + k > 0$, $r(d + k) < dk \iff r < dk/(d + k)$.
2. 4096×4096: $dk = 16\,777\,216$, $d + k = 8192$; límite 2048. LoRA ahorra si $r < 2048$.
3. 1024×4096: $dk = 4\,194\,304$, $d + k = 5120$; límite 819,2. Como r es entero, LoRA ahorra si $r \leq 819$.

Resultado. $r < dk/(d + k)$**:** $r < 2048$ **para** 4096×4096 **y** $r \leq 819$ **para** 1024×4096**.**

Comprobación. con $d = k$ el límite es $d/2$, que da 2048 para $d = 4096$. En la práctica se usan rangos de 4 a 64 porque la actualización útil tiene espectro concentrado (corolario 5.12), el estado del optimizador queda mínimo y un rango pequeño regulariza.

Bibliografía

[1] A. Aghajanyan, L. Zettlemoyer y S. Gupta. Intrinsic dimensionality explains the effectiveness of language model fine-tuning. En *Proceedings of ACL-IJCNLP*, 2021. https://arxiv.org/abs/2012.13255

[2] Y. Bai *et al.* Constitutional AI: Harmlessness from AI feedback. Anthropic, 2022. https://arxiv.org/abs/2212.08073

[3] C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006. §4.4, «The Laplace Approximation».

[4] G. Neubig. 11-711: Advanced Natural Language Processing. Carnegie Mellon University, otoño de 2024. https://www.phontron.com/class/anlp-fall2024/

[5] D. Yang y Y. Choi. CS224N: Natural Language Processing with Deep Learning. Stanford University, invierno de 2026. Clase 8, «Post-training (RLHF, SFT, DPO)»; clase 9, «Efficient Adaptation (Prompting + PEFT)»; clase invitada de J. Schulman, «Tinker and LoRA Without Regret». https://web.stanford.edu/class/cs224n/

[6] T. Hashimoto y P. Liang. CS336: Language Modeling from Scratch. Stanford University, primavera de 2025. Clase 15, «Alignment – SFT/RLHF». https://cs336.stanford.edu/spring2025

[7] Stanford CS336. Assignment 1 (basics): Building a Transformer LM. Primavera de 2026. §3.5, problema `transformer_accounting`. https://github.com/stanford-cs336/assignment1-basics

[8] Stanford CS336. Assignment 5 (alignment): Alignment and Reasoning RL. Primavera de 2025. §4.2, problemas `tokenize_prompt_and_output`, `masked_normalize` y `sft_microbatch_train_step`. https://github.com/stanford-cs336/assignment5-alignment

[9] Stanford CS336. Assignment 5 Supplement: Instruction Tuning and RLHF. Primavera de 2026. §4.2, problema `data_loading`. https://github.com/stanford-cs336/assignment5-alignment

[10] M. P. Deisenroth, A. A. Faisal y C. S. Ong. *Mathematics for Machine Learning*. Cambridge University Press, 2020. Versión en línea de 2024. Cap. 4 «Matrix Decompositions», §4.6 y ejercicio 4.10. https://mml-book.github.io/

[11] T. Dettmers, A. Pagnoni, A. Holtzman y L. Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. En *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. https://arxiv.org/abs/2305.14314

[12] C. Eckart y G. Young. The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3):211–218, 1936.

[13] Gemini Team, Google. Gemini: A family of highly capable multimodal models. Informe técnico, 2023. https://arxiv.org/abs/2312.11805

[14] I. Goodfellow, Y. Bengio y A. Courville. *Deep Learning*. MIT Press, 2016. §7.1.1, «L^2 Parameter Regularization». https://www.deeplearningbook.org/

[15] J. He, C. Zhou, X. Ma, T. Berg-Kirkpatrick y G. Neubig. Towards a unified view of parameter-efficient transfer learning. En *International Conference on Learning Representations (ICLR)*, 2022. https://arxiv.org/abs/2110.04366

[16] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan y S. Gelly. Parameter-efficient transfer learning for NLP. En *International Conference on Machine Learning (ICML)*, 2019. https://arxiv.org/abs/1902.00751

[17] J. Howard y S. Ruder. Universal language model fine-tuning for text classification. En *Proceedings of ACL*, 2018. https://arxiv.org/abs/1801.06146

[18] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang y W. Chen. LoRA: Low-rank adaptation of large language models. En *International Conference on Learning Representations (ICLR)*, 2022. https://arxiv.org/abs/2106.09685

[19] D. Jurafsky y J. H. Martin. *Speech and Language Processing*, 3.ª ed., borrador del 19 de agosto de 2026. Cap. 8, «Post-training»; cap. 11, «Information Retrieval and RAG». https://web.stanford.edu/~jurafsky/slp3/

[20] J. Kirkpatrick *et al.* Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017. https://arxiv.org/abs/1612.00796

[21] B. Lester, R. Al-Rfou y N. Constant. The power of scale for parameter-efficient prompt tuning. En *Proceedings of EMNLP*, 2021. https://arxiv.org/abs/2104.08691

[22] C. Li, H. Farkhoor, R. Liu y J. Yosinski. Measuring the intrinsic dimension of objective landscapes. En *International Conference on Learning Representations (ICLR)*, 2018. https://arxiv.org/abs/1804.08838

[23] X. L. Li y P. Liang. Prefix-tuning: Optimizing continuous prompts for generation. En *Proceedings of ACL-IJCNLP*, 2021. https://arxiv.org/abs/2101.00190

[24] M. McCloskey y N. J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. *Psychology of Learning and Motivation*, 24:109–165, 1989.

[25] L. Ouyang *et al.* Training language models to follow instructions with human feedback. En *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. https://arxiv.org/abs/2203.02155

[26] V. Sanh *et al.* Multitask prompted training enables zero-shot task generalization. En *International Conference on Learning Representations (ICLR)*, 2022. https://arxiv.org/abs/2110.08207

[27] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai y Q. V. Le. Finetuned language models are zero-shot learners. En *International Conference on Learning Representations (ICLR)*, 2022. https://arxiv.org/abs/2109.01652

[28] A. Zhang, Z. C. Lipton, M. Li y A. J. Smola. *Dive into Deep Learning*, versión 1.0.3. Cambridge University Press, 2023. §14.2, «Fine-Tuning». https://d2l.ai/

[29] C. Zhou *et al.* LIMA: Less is more for alignment. En *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. https://arxiv.org/abs/2305.11206